DataEnvGym：以学生反馈为信号的教师环境中的数据生成智能体

DataEnvGym: Data Generation Agents in Teacher Environments with Student Feedback

Zaid Khan、Elias Stengel-Eskin、Jaemin Cho、Mohit Bansal

北卡罗来纳大学教堂山分校（UNC Chapel Hill）

{zaidkhan, esteng, jmincho, mbansal}@cs.unc.edu · DataEnvGym.github.io

ICLR 2025 · arXiv:2410.06215v3 [cs.CL] · 原文 28 页

> 中文阅读版说明：本文件依据 ICLR 2025 论文逐节翻译，保留原有章节顺序、表格数值与图注位置。图 1—5、13、14、17、18 保留原图并附中文图注；表 1—8 以中文重排，数值照录原文。模型名、数据集名、环境名（OPEN-ENDED、SKILL-LIST、SKILL-TREE）与参考文献条目保持英文，便于检索。附录中的提示词模板保留 Jinja 变量与代码结构，仅翻译说明性文字。

摘要

当前，为教模型而创建训练数据的过程由人主导：人手动分析模型弱点，并规划如何构造能改进学生模型的数据。近来用大语言模型（LLM）充当标注者的做法降低了人工标注负担，但仍需人来解读评测反馈，并操控 LLM 生成学生所需的数据。若能创造自主的数据生成智能体——也就是“教师”——来自动化这一劳动密集型过程，当然理想；但这需要能够模拟“反馈驱动、迭代、闭环”数据创造过程的环境。为了对此类智能体及其模块进行快速、可扩展的测试，我们提出 DATAENVGYM，一个面向数据生成智能体的教师环境测试床。DATAENVGYM 把数据生成建模为序贯决策任务：智能体由数据生成策略（生成训练数据创造计划）与数据生成引擎（把计划转化为数据）组成，处在一个能提供学生反馈的环境之中。智能体的最终目标是提升学生模型的表现。学生模型在生成数据上被迭代地训练与评估，每次迭代后把反馈（以错误或薄弱技能的形式）报告给智能体。作为一个通用测试床，DATAENVGYM 包含多种教师环境实例，覆盖状态表示与动作空间的三个结构化层级，并提供不同程度的脚手架支持。结构化程度更高的环境建立在自动推断的技能之上，对课程具有更高的可解释性与控制力。我们支持在文本、图像与动作四类多样任务（数学、编程、视觉问答、工具使用）上开发与测试数据生成智能体，并测试了多种学生模型与教师模型。我们发现，教学环境中的示例智能体能够在多种任务与设置下迭代地改进学生。此外，我们表明这些环境可以教授不同技能层级，也可用于测试关键模块的变体，为改进数据生成智能体、引擎与反馈机制的未来工作指明了方向。

项目主页：https://DataEnvGym.github.io。

1 引言

通过构造额外的、针对当前模型弱点的训练数据来改进一个已训练模型，是研究者与工程师一项重要而频繁的任务。例如，指令微调与对齐领域的过往工作发现，模型可以借助额外的任务特定训练样例得到改进（Touvron et al., 2023; Ding et al., 2023; Zhou et al., 2024; Chia et al., 2024; Wang et al., 2023b; Shimabucoro et al.,

2024）。然而，当前的模型改进过程在很大程度上由人驱动：人依据评测尝试识别模型弱点，凭直觉与启发式构造针对弱点的数据，在数据上训练更新后的模型，再依据新模型的表现修订数据（Iyer et al., 2022; Longpre et al., 2023; Shao et al., 2024）。这一过程所涉及的劳动与重复，强烈促使我们创造数据生成智能体，以全部或部分地自动化"造数据教学生模型"的过程。在我们此前的工作 EnvGen（Zala et al., 2024）中，我们自动化了简单游戏实例的设计，使小型强化学习模型在预定义技能上得到改进。本文要问：这一做法如何扩展到更宏大、更真实的开放式任务，例如视觉问答、数学或代码生成？然而与游戏任务不同，目前尚无模拟器或环境可作为测试床，用来为这些更复杂的任务开发自动化教师智能体，甚至无法评估自动化这一迭代式数据生成与模型改进过程所需的各个组件的有效性。

我们提出 DATAENVGYM：一个由可参数化教师环境构成的测试床（gym），用于开发自主数据生成智能体（即教师）。其目标是：以学生弱点的表示（如错误、推断出的薄弱技能）为条件，生成有针对性的训练数据，从而迭代地改进学生模型。我们把该任务刻画为教师智能体与学生模型之间的迭代交互：教师智能体创造训练数据，学生模型由此得到改进。正如游戏环境可用于评估传统强化学习（RL）中的游戏智能体，DATAENVGYM 的模块化环境（参见第 2.1 节）也让我们能够针对给定学生模型测试数据生成智能体。在这些环境中，数据生成智能体（教师）执行多轮数据创造，并在每次迭代后收到以学生表现形式给出的学生反馈，这就是教师的奖励信号。我们提供数据生成、训练与评估模块，最终表现以学生模型的提升来衡量。DATAENVGYM 的概览见图 1。首先，环境向智能体提供状态 s_t，其中包含当前学生模型 m_t 的错误信息（参见图 1(a)）。随后，智能体的数据生成策略 π 预测构成训练数据生成计划的动作（参见图 1(b)）：$p_t \sim \pi(p_t|s_t)$。接着，智能体的数据生成引擎执行该计划，创造训练数据 d_t（参见图 1(c)）。所创造的数据点随后用于训练更新后的学生模型（参见图 1(d)）：$m_{t+1} = \text{Train}(m_t, d_t)$。更新后的学生模型被重新评估，以产生下一轮迭代的状态 s_{t+1}，并向智能体提供反馈（以学生表现的形式）。DATAENVGYM 以可泛化的方式设计，支持跨多个任务、面向多样智能体的数据创造，覆盖多模态（视觉问答）与纯文本（数学、编程）任务。

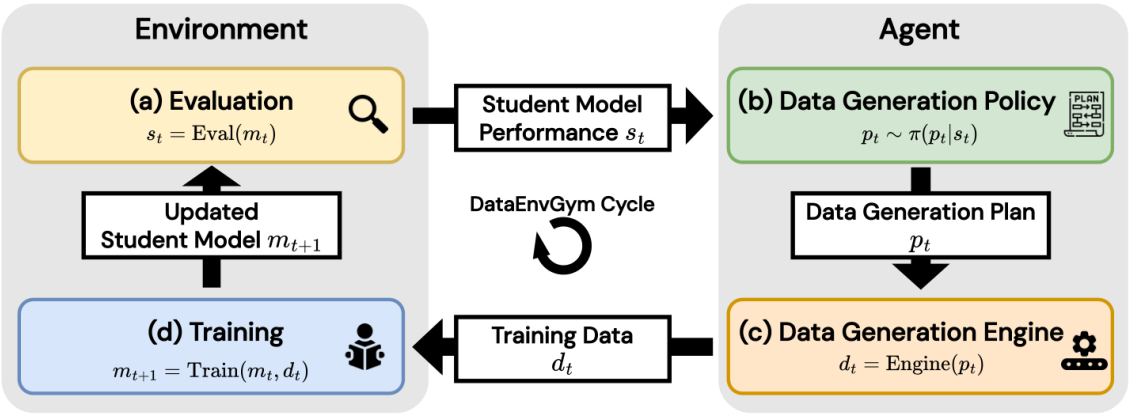


图 1：DATAENVGYM 概览——一个面向数据生成智能体的新测试床。环境（左）由学生模型的 (a) 评估与 (d) 训练组成。数据生成智能体（右）接收编码当前学生模型表现的状态，并通过 (b) 数据生成策略先制定计划、再经 (c) 数据生成引擎执行计划，从而提供训练数据以改进学生模型。

DATAENVGYM 的模块化设计支持多种数据生成环境实例。我们提供三种 DATAENVGYM 环境实现，以及各自所需的智能体模块。它们在向智能体提供的状态表示与动作空间上有所不同，从开放式（直接依据逐样例模型预测生成数据）到更结构化（依据基于技能的、中间模型进展的层级表示生成数据）。首先，在 OPEN-ENDED 环境（参见图 2(a)）中，状态表示是学生模型错误的无结构列表。该环境的动作空间同样无结构（即

开放式），一个动作就是生成某一组特定数据点；也就是说，智能体直接从错误中推断何种数据会有帮助，然后直接生成该数据。这与人类开发者形成对比：人通常采用更偏向技能的方法，把表现拆解为技能特定的指标，以决定在何处补充数据。基于技能的开发有三个明显优势：它为智能体提供控制数据生成过程的结构化方式；它围绕易于把握的技能组织数据生成，使过程更具可解释性；它支持人机互操作与课程控制，人或其他模型都可以指定希望模型改进的技能。

基于这些优势，我们认为在某些情况下，技能结构化的智能体可能更可取，甚至是必需的。因此，DATAENVGYM 也支持基于技能的教学与学习。具体而言，DATAENVGYM 包含 SKILL-LIST 环境（参见图 2(b)）：技能发现模块首先使用大语言模型（LLM）从训练数据中自动推断出人类可解释的技能。这产生更结构化的状态表示（即技能特定表现的报告），并使智能体的任务更可解释，因为它能获得关于学生模型在哪些技能上吃力的显式反馈。与 OPEN-ENDED 环境一样，SKILL-LIST 环境要求智能体直接生成数据。虽然 SKILL-LIST 环境通过在状态表示中加入技能，比 OPEN-ENDED 环境提供了更多结构与可解释性，但两者的动作空间都很细粒度、自由度很高。因此，SKILL-LIST 的输入空间虽更结构化、更可解释，其输出空间却并非如此。为了给智能体更结构化的输出，我们还加入了一个把动作空间结构化为固定、较粗粒度动作的环境。为此，在 SKILL-TREE 环境（参见图 2(c)）中，我们把技能抽象为一种基于树的表示，称为“技能森林”（skill forest）。这里，技能被分层组织为技能树：父技能是根节点，子技能是子节点（示例见图 3）。这种层级框架允许发现新的、更细粒度的子技能，并把智能体的任务简化为两种动作之间的二元选择：explore 动作通过添加新的子技能来生长技能树；exploit 动作重新平衡技能树，把更多数据分配给已有子技能。这一划分旨在帮助智能体优先处理重要技能（为过去有助于提升表现的技能生成更多数据），同时平衡技能层级中广度与深度的竞争压力（通过添加新子技能来拓宽技能树）。我们的 DATAENVGYM 测试床不仅为所有这些环境与组件提供

默认实现，也便于测试替代实现；例如，改进的技能发现实现或另一种数据结构都可以轻松接入，并依据下游学生表现进行测试。这些环境的输入空间与动作空间总结见图 3。

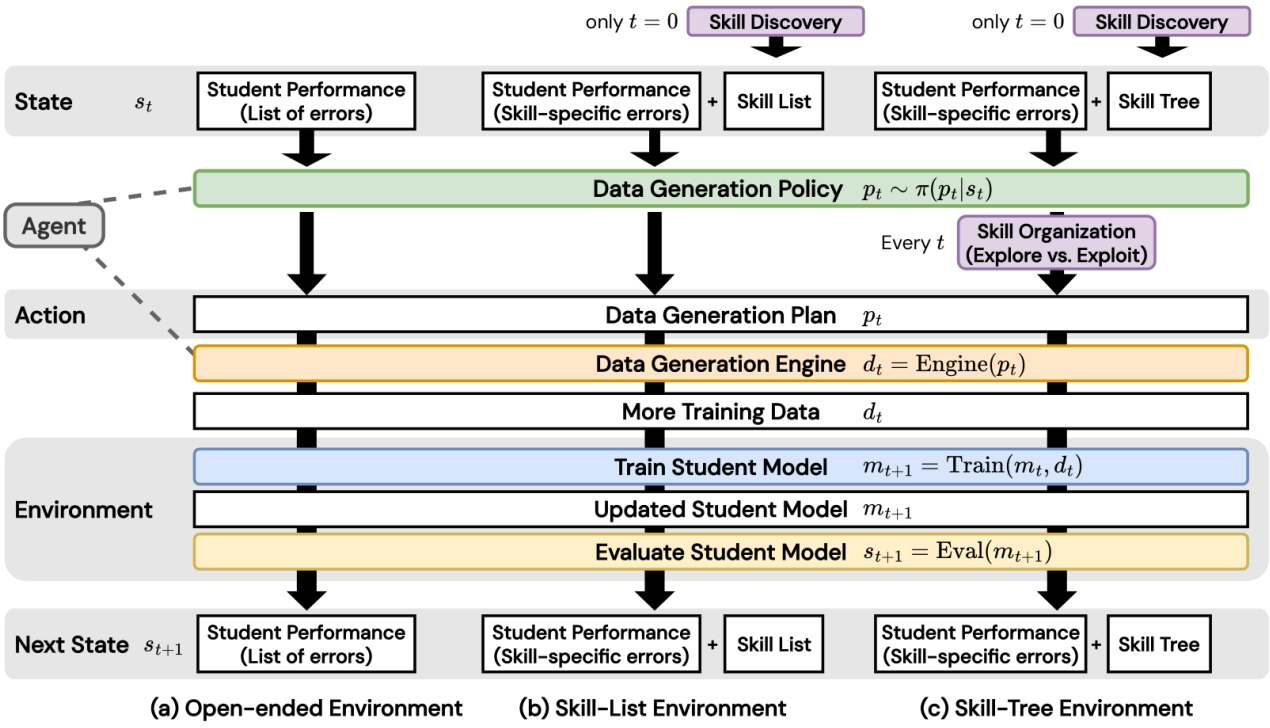


图 2：第 2 节所述三种 DATAENVGYM 环境实例的示意。(a) OPEN-ENDED 环境中，状态表示为逐样例正确率列表，数据生成策略直接据此制定数据生成计划。(b) SKILL-LIST 环境中，状态表示为已分类的技能列表及学生模型在各技能上的表现；其数据生成计划允许策略优先处理薄弱技能。(c) SKILL-TREE 环境中，状态表示为包含技能—子技能关系信息的技能树森林，其数据生成策略对每个技能在两种动作间选择：explore（生长技能树）与 exploit（重新平衡技能树）。

我们在 DATAENVGYM 的教学环境中，以若干基线智能体（数据生成策略与数据生成引擎的组合）为示例，在不同领域（数学、编程、视觉问答）以及不同学生模型与教师模型上进行了基准测试。总体而言，我们发现所提供的示例智能体已经能在 DATAENVGYM 教学环境中训练时提升学生表现；训练后，学生相较于起点（即在环境中训练之前）有一致的提升。跨环境平均看，学生在 GQA 上提升 4.43%（绝对准确率），MATH 上提升 4.82%，LiveCodeBench 上提升 1.80%，NaturalBench 上提升 15.17%，MnMs 上提升 20.81%。此外，我们发现示例智能体能够利用学生反馈来迭代改进学生：我们比较利用学生反馈状态的基线策略（“With State”策略）与不利用反馈的策略（“No State”策略），发现以反馈状态为条件对成功的数据生成至关重要：“With State”在 OPEN-ENDED 中比“No State”高 3.5%，在 SKILL-LIST 中高 2.05%，在 SKILL-TREE 中高 1.08%。我们还表明，依据课程需要多灵活、多可控（多可解释），某些环境会让教师改进学生变得更困难。此外，我们表明环境可以教授不同技能（依据技能频率与题目难度），并可用于测试关键模块的变体，例如技能发现模块。最后，我们给出学生模型在训练前后预测的定性示例。总体而言，DATAENVGYM 是一个用于开发与评估数据生成智能体、引擎与反馈机制的通用测试床，为未来改进自动化模型改进中的这些关键要素奠定了基础。

2 DATAENVGYM 的环境与智能体

我们在 DATAENVGYM 中提供三类（环境，智能体）组合，给予数据生成智能体多层次的结构，对应不同层级的可解释性。智能体由两个模块组成：数据生成策略 π（制定数据生成计划 $p_t \sim \pi(p_t|s_t)$）与数据生成引擎 Engine（执行计划以产生训练数据 $d_t = \text{Engine}(p_t)$；参见第 2.2.2 节）。策略与计划都可随智能体所处环境而变化，因为环境向智能体提供的可供性（affordances）定义了其动作空间。环境封装了若干模块，包括学生模型 m_t、训练器（给定生成数据训练学生模型，$m_{t+1} = \text{Train}(m_t, d_t)$）与评估器（评估更新后的学生模型并输出其表现，$s_{t+1} = \text{Eval}(m_{t+1})$；参见第 2.1.1 节）。

如表 1 所总结，某些环境还包含额外模块，用于通过自动技能发现（第 2.1.2 节）与组织（第 2.1.3 节）生成技能特定的训练样例。基于技能的结构给智能体带来三个明显优势：第一，它为智能体提供可供性，以控制所生成数据的目标性或多寡性（即调节数据在多大程度上针对单一技能，还是广泛改进多种技能）；第二，当技能对人可解释时，基于技能的智能体支持人机互操作与人在回路控制，人可以影响数据生成过程（例如指定要改进的技能），或在需要时实际替代智能体（反之亦然）；第三，基于技能的改进带来可解释的智能体行为——用户可以观察正在为哪些技能生成数据、训练在何处最有效。我们为三个不同任务提供上述全部组件：数学、视觉问答（VQA）与编程。

（1）OPEN-ENDED 环境。 OPEN-ENDED 环境如图 2(a) 所示，向数据生成智能体提供最简单的状态结构，是约束最少的环境。*状态表示：* 状态表示为学生评估预测的列表。智能体必须从该列表推断何种数据最能帮助学生，把列表中的错误直接映射为期望的未来数据点。*动作空间：* OPEN-ENDED 环境提供的动作空间直接指定要生成的数据点，即智能体需要直接为每个数据点生成规格，没有任何辅助动作来结构化生成过程。

（2）SKILL-LIST 环境。 SKILL-LIST 环境（图 2(b)）要求教师（即数据生成智能体）教授一组特定技能。*状态表示：* SKILL-LIST 环境在训练数据上归纳出任务所需技能（参见第 2.1.2 节），并报告学生在各技能上的表现。智能体策略的输入是按技能划分的评估预测列表。这告诉智能体：在需要该技能的问题上犯了哪些错误。*动作空间：* 动作空间与 OPEN-ENDED 共享。

（3）SKILL-TREE 环境。 SKILL-TREE 环境如图 2(c) 所示，把数据生成与数据控制解耦，为动作空间增加结构，使其策略不再直接生成数据规格，而只规定要生成多少数据、面向哪些子技能。这约束了动作空间，并为智能体提供额外脚手架。*状态表示：* 状态表示的表面形式与 SKILL-LIST 环境共享。但 SKILL-TREE 环境还维护一个由技能树构成的底层技能森林，每棵树是某技能及其子技能的层级表示（细节见第 2.1.3 节，示例见图 3）。因此，尽管输入相似（技能名与学生在各技能上的表现），实际技能不同于 SKILL-LIST 环境——后者没有任何层级结构。*动作空间：* SKILL-TREE 环境为智能体提供更结构化的动作空间。每轮迭代中，智能体不再直接生成数据，而是对每个技能（及其对应技能树）选择：exploit，即通过为已有技能重新平衡技能树来利用现有技能集（把数据预算重新分配给其子技能）；或 explore，即通过创造新子技能来生长技能树。动作作用于技能树，产生一棵新的技能树——要么加入了新子技能，要么改变了分配给各子技能的数据量。随后，数据生成引擎消费每棵技能树，为每棵树内的每个子技能生成计划数量的训练数据。

下面我们描述 DATAENVGYM 中实例化的各组成模块（第 2.1 节）与数据生成智能体（第 2.2 节）。

2.1 环境模块

2.1.1 训练器与评估器

给定来自数据生成引擎的训练数据 d_t，训练器执行一次训练运行（即在数据集上训练一定步数），更新学生模型：$m_{t+1} = \text{Train}(m_t, d_t)$。随后，评估器测试学生模型并输出其表现：$s_{t+1} = \text{Eval}(m_{t+1})$。

基线实现。 我们使用 Transformers（Wolf et al., 2020）库进行监督微调。数据采用 Alpaca（Taori et al., 2023）的指令微调格式，使用标准语言建模损失。评估时，我们使用所测试数据集的标准训练/测试划分。训练过程的更多细节（包括学习率、优化器等超参数）见附录 B.1。

2.1.2 技能发现

SKILL-LIST 与 SKILL-TREE 环境有一个技能发现模块：它接收一组训练样本 d_t，返回解决这些样例所需的技能集合；在训练开始时（$t = 0$），环境使用该模块在验证集上发现一组技能。或者，环境也可以由一组用户指定的目标技能来参数化。技能发现模块会为验证集中每个已评估的学生预测分配一个发现的技能标签。技能列表与已评估的学生预测直接由 SKILL-LIST 环境消费。在 SKILL-TREE 环境中，技能则作为技能组织模块的输入。

基线实现。 为发现技能，我们的技能发现模块基线实例采用两阶段方法，遵循 Didolkar et al. (2024)。第一，为每个任务实例分配一个特定技能，使用一个模板要求 LLM 识别解决该问题所需的高层技能。第二，用另一个模板把这些技能聚合为类别，要求 LLM 对相似技能分组。具体而言，我们向 LLM 提供完整技能列表，要求它把技能穷尽地划分为互斥类别。

在 SKILL-TREE 环境中，我们用同样的过程以层级方式为已有技能提出子技能。例如，给定技能“Algebra”，子技能可能是“Solving Linear Equations”或“Polynomial Factorization”。实现上，我们向 LLM 提供一个技能及该技能已有的子技能，指示它提出属于给定技能且不重复的子技能。必要时，我们可以在已有技能之上重复此过程，为每个技能归纳出一组子技能。LLM 提示词见附录 B.2。

2.1.3 技能组织

为了通过自适应地扩展学生可完成的技能集合来解决复杂问题，把技能组织成某种层级是很自然的。在 SKILL-TREE 环境中，技能组织模块接收一组技能，输出“技能树”森林——一种有组织的层级结构，编码技能并存储其元数据（例如分配给每个技能多少数据）。这就是 SKILL-TREE 环境中的状态 s_t。图 3 展示了一个技能树示例。

基线实现。 在 SKILL-TREE 环境中，技能森林以逐步细化的粒度刻画学生的熟练度：每棵树的根对应一个高层技能领域，子节点对应子技能。森林中的每棵树包含子技能的关键信息，包括分配给每个子技能的训练数

据量（即数据分配）以及学生在每个子技能训练划分上的表现。注意，技能森林的具体实现留给用户；DATAENVGYM 提供默认版本，但也可接入其他实现。

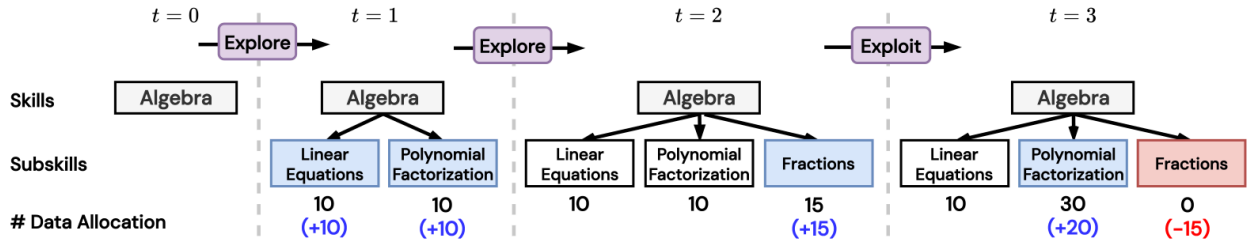


图 3：SKILL-TREE 环境中 MATH 任务“Algebra”技能的技能树随时间更新示例。数据生成策略（第 2.2.1 节）从空的单节点开始，迭代地在 explore（生长技能树）与 exploit（重新平衡技能树）之间选择动作。随后，技能组织模块（第 2.1.3 节）相应地添加/删除子技能，并为每个子技能重新分配训练数据。

2.2 数据生成智能体模块

2.2.1 数据生成策略

数据生成策略 π 以学生表现状态 s_t 为输入（OPEN-ENDED 环境为逐样例错误列表；SKILL-LIST 环境为技能特定错误与技能列表；SKILL-TREE 环境为技能特定错误与技能树），输出作为动作的数据生成计划（即数据生成引擎的输入）：$p_t \sim \pi(p_t|s_t)$。在 OPEN-ENDED 与 SKILL-LIST 环境中，数据生成计划是训练数据规格的列表，每个待产生训练数据对应一条。数据生成引擎会把训练数据渲染或格式化为适合指令微调的格式。在 SKILL-TREE 环境中，我们塑造了动作空间，提供两个离散动作：explore 与 exploit；注意，进一步的动作可以很容易地加入。explore 动作通过添加子技能来生长技能树；exploit 动作改变已有子技能的数据分配。

基线实现。 对于 OPEN-ENDED 与 SKILL-LIST 环境，我们用 LLM 驱动策略：把状态口语化后交给 LLM，并提示它产生相应动作。对于 SKILL-TREE 环境，我们实现了一个策略，通过交替 explore 与 exploit 动作，把技能树生长到固定规模并维持均匀的数据分布。细节见附录 B.6。

2.2.2 数据生成引擎

数据生成引擎的职责是依据策略给出的数据生成计划产生训练样例：$d_t = \text{Engine}(p_t)$。这些训练样例将用于教学生。由于每个环境给智能体提供的动作空间不同，对应的数据生成引擎也不同。具体而言，对于 OPEN-ENDED 与 SKILL-LIST 环境，数据生成引擎接收的动作形式是“要生成的数据点”（因为策略的动作空间无结构），并格式化相应样例（例如对 GQA，它用 T2I 模型生成图像）。OPEN-ENDED 与 SKILL-LIST 生成器可访问任务以及一份待渲染为训练数据的样例列表。对于动作空间为 {explore, exploit} 的 SKILL-TREE 环境，数据生成引擎必须首先解释这些动作。每个动作会触发对技能树的一次修改。explore 动作调用子技能发现流水线，通过添加子技能来生长技能树。发出 exploit 动作时，智能体必须指定如何改变每个子技能的数据分配（预算）；执行该动作即相应调整技能树中存储的预算。最后，数据生成引擎消费技能树，为每个子技能生成计划数量的数据。

基线实现。 对所有任务（数学、VQA、编程），我们使用 LLM（GPT-4o）生成训练数据。对于数学题，我们用 LLM 生成题目，每题包含问题、逐步解答与最终答案。对于 VQA 任务，我们首先用 LLM 生成与给定任务/技能/子技能对齐的图像描述，再用文本到图像模型把描述转换为图像；然后指示 LLM 为给定任务/技能/

子技能生成指定数量的独特问题。对于编程，我们分两阶段生成数据：依据子技能信息与关于期望格式的详细说明生成题目和起始代码，再用一次独立的 LLM 调用求解。各环境生成器的细节见附录 B.3，各任务生成的训练样例见附录 C。

表 1：DATAENVGYM 基线环境总结。不同组件决定每轮迭代如何生成训练样例。

环境	训练器/评估器（2.1.1）	技能发现（2.1.2）	技能组织（2.1.3）
OPEN-ENDED	✓	-	-
SKILL-LIST	✓	✓	-
SKILL-TREE	✓	✓	✓

3 实验

我们在四个领域实验 DATAENVGYM 环境：视觉问答、数学、编程与工具使用。视觉问答使用 GQA（Hudson & Manning, 2019）与 NaturalBench（Li et al., 2024）；数学使用 MATH（Hendrycks et al., 2021）；编程使用 LiveCodeBench（Jain et al., 2024）；工具使用使用 MnMs（Ma et al., 2024）。对大多数实验（第 3.1 节报告），我们从指令微调模型而非基座模型出发，因为我们认为这是更现实、更具挑战性的设置：从指令微调模型出发是应用中的标准做法，而这些模型已经过大量任务特定数据的后训练。对于 GQA 与 NaturalBench，学生模型为 PaliGemma-3b-pt-224（Beyer et al., 2024），教师智能体策略为 GPT-4o（OpenAI, 2024），并用 SDXL-Turbo（Sauer et al., 2023）做 T2I 生成。对于 MATH，学生为 Gemma-2-2B-Instruct（Gemma Team, 2024）。对于 LiveCodeBench 与 MnMs，学生为 Llama-3-8B-Instruct（Llama Team, 2024）。所有领域的数据均由 GPT-4o 生成。注意，我们使用的学生模型通常已经擅长目标任务，因此难以进一步提升。对每个领域，我们按以下标准选择学生模型：1）学生应足够强，能完成给定任务（例如 LiveCodeBench 对 Gemma2-2B 学生过于困难）；2）学生不应已被大量后训练到几乎不可能再提升（例如 Llama3-8B 已在数学上广泛训练，无论数据生成智能体如何，额外训练都难以再改进）。细节见附录 B.7（验证/测试划分）与附录 B.3（数据生成）。工具使用（MnMs）与额外多模态实验（NaturalBench）在第 3.2 节详述。

3.1 主要结果：VQA、数学、编程

表 2 给出 DATAENVGYM 内示例环境实例的结果。这里，我们比较学生在跨环境多步训练轨迹前后的表现，使用不同的数据生成策略。对每种设置，我们报告相对于在 DATAENVGYM 中训练前学生模型的相对增益或损失。另请注意，这里使用的模型已在大规模数据集（包括任务特定数据集）上指令微调，因此获得进一步提升尤其困难。

LLM 策略可以利用状态信息为学生提供更好的训练数据。 在“No State”设置下训练的学生，通常不如在“With State”设置下训练的学生。这在不同环境中都成立：OPEN-ENDED 环境差异最大（3.5%），SKILL-TREE 环境差异最小（1.08%）。在 LiveCodeBench 上，没有状态信息的策略完全无法提升学生；而在 MATH 上，没有状态信息的策略在所有环境中仍能提升学生。SKILL-TREE 环境为教师提供的支持在 GQA 上尤其稳健：没有状态信息的策略几乎达到有状态信息策略的相同表现。然而，若没有 SKILL-TREE 的结构化动作，去掉状态信息实际上会损害 GQA 表现——OPEN-ENDED 与 SKILL-LIST 的“No State”设置相较基线略有下降。对这两个环境，“With State”都能改进学生模型。综合来看，这些结果凸显了状态信息的重要性。

某些环境中的教学比其他环境更容易。 固定学生与任务（即看表 2 各列的“With State”条目），教师通常在约束较少的 OPEN-ENDED 环境中取得最高的学生表现，因为不要求教授特定技能集。但这里可能存在领域特定效应：SKILL-TREE 环境中的教师在多模态 GQA 数据集上表现最好（+5.58%），而在 MATH 上恰好相反，OPEN-ENDED 环境中的教师表现最好（+7.66%）。这些差异可能与约束程度有关：在 OPEN-ENDED 环境中，教师可以在无约束下产生任何数据；而在技能结构化环境中，我们要求教师沿指定技能改进学生。这可能是更困难的任务，因为它可能对教师施加次优约束，即教师可能难以教授指定技能；而在无约束的 OPEN-ENDED 环境中，教师可能隐含地识别并教授自己最擅长教的技能。不过，无约束教学（错误与生成数据之间没有明确联系）在实践中未必总是有用，例如当用户想控制训练、对某些技能做针对性改进时。

表 2：DATAENVGYM 环境中的智能体能够跨任务与教学环境改进学生。 ∗“No State”在 OPEN-ENDED 与 SKILL-LIST 环境中相同，因为二者只在状态表示上不同，其“No State”策略（不以状态为条件）完全一致。

设置/环境	GQA（PaliGemma 3B）	MATH（Gemma2 2B）	LiveCodeBench（Llama3 8B）	平均提升
教学前	44.18	15.78	16.50	-
OPEN-ENDED 环境				
+ No State∗ $\pi(p_t\|\cdot)$	43.48 (-0.70%)	19.78 (+4.00%)	16.50 (-0.00%)	(+1.10%)
+ With State $\pi(p_t\|s_t)$	47.90 (+3.72%)	23.44 (+7.66%)	18.91 (+2.41%)	(+4.60%)
SKILL-LIST 环境				
+ No State∗ $\pi(p_t\|\cdot)$	43.48 (-0.70%)	19.78 (+4.00%)	16.50 (-0.00%)	(+1.10%)
+ With State $\pi(p_t\|s_t)$	48.18 (+4.00%)	19.48 (+3.70%)	18.25 (+1.75%)	(+3.15%)
SKILL-TREE 环境				
+ No State $\pi(p_t\|\cdot)$	49.53 (+5.35%)	17.15 (+1.37%)	16.50 (-0.00%)	(+2.24%)
+ With State $\pi(p_t\|s_t)$	49.76 (+5.58%)	18.90 (+3.12%)	17.75 (+1.25%)	(+3.32%)

我们对所有模型训练固定步数，并在固定迭代次数后终止回合，使用验证准确率选择学生准确率最高的训练数据迭代。对每个环境与领域，我们报告两个值：第一，第 2.2.1 节所述教师策略基线实现所取得的学生表现提升；该策略接收状态 s_t，即 $\pi(p_t|s_t)$。我们称此设置为 With State。第二，同一策略在不以状态信息为条件时（即从 $\pi(p_t|\cdot)$ 采样动作、不以 s_t 为条件）所取得的学生表现提升。对 OPEN-ENDED 环境策略，我们把学生错误列表替换为随机训练样本；对 SKILL-LIST 策略同样处理；对 SKILL-TREE 策略，我们以相等概率采取 explore 与 exploit 动作。我们称此设置为 No State。

3.2 额外的多模态与工具使用基准

本节展示在额外困难的多模态与工具使用任务上的结果：NaturalBench（Li et al., 2024，表 3）与 MnMs（Ma et al., 2024，表 4）。MnMs 与 NaturalBench 包含准确率之外的细粒度指标，能更细致地观察不同方法生

成的数据如何影响学生模型。在 MnMs 与 NaturalBench 上，以任务和学生错误为条件，都能让教师生成更有效的训练数据（比较表 3、表 4 中的“With State”与“No State”）。

在 NaturalBench 上，以学生状态为条件生成的训练数据在所有指标上都更有效地改进学生。“Group Score”考察模型在四个测试同一概念的视觉问答对上的一致性：不以状态为条件生成的数据带来的 Group Score 提升为零，而以状态为条件则带来 Group Score 提升。

MnMs 是一个工具使用基准，需要修改技能发现机制，因为标准流水线应用于 MnMs 时产生的技能与所需底层工具耦合不紧。对 MnMs，我们把每种工具组合视为一个“技能”。这导致工具使用的“技能”数量非常多。为在固定预算下生成数据，我们必须采样一部分技能（工具组合）来生成数据。我们尝试了多种采样方法，发现最有效的是随机采样学生尚未接近完美准确率的技能子集。

表 3：NaturalBench（Li et al., 2024）在教学前后使用 OPEN-ENDED 与 SKILL-LIST 环境的性能指标。

设置	Question Score	Image Score	Binary Score	Group Score	平均提升
教学前	3.92	3.92	48.53	0.00	-
OPEN-ENDED 环境					
+ With State $\pi(p_t \mid s_t)$	26.00 (+22.08%)	29.00 (+25.08%)	61.00 (+12.47%)	6.00 (+6.00%)	(+16.41%)
+ No State $\pi(p_t \mid \cdot)$	11.00 (+7.08%)	9.00 (+5.08%)	51.50 (+2.97%)	0.00 (+0.00%)	(+3.78%)
SKILL-LIST 环境					
+ With State $\pi(p_t \mid s_t)$	25.00 (+21.08%)	26.00 (+22.08%)	59.50 (+10.97%)	2.00 (+2.00%)	(+14.03%)
+ No State $\pi(p_t \mid \cdot)$	11.00 (+7.08%)	9.00 (+5.08%)	51.50 (+2.97%)	0.00 (+0.00%)	(+3.78%)

表 4：MnMs（Ma et al., 2024）在教学前后使用 OPEN-ENDED 与 SKILL-LIST 环境的性能指标。

设置	Plan Accuracy	Tool Precision	Tool Recall	Tool F1	平均增益
教学前	37.30	73.74	68.18	65.41	-
OPEN-ENDED 环境					
+ With State $\pi(p_t \mid s_t)$	72.00 (+34.70%)	87.32 (+13.98%)	82.62 (+14.44%)	82.25 (+16.84%)	(+19.99%)
+ No State $\pi(p_t \mid \cdot)$	62.13 (+24.83%)	87.94 (+14.60%)	87.09 (+12.91%)	80.51 (+15.10%)	(+16.86%)
SKILL-LIST 环境					
+ With State $\pi(p_t \mid s_t)$	75.96 (+38.66%)	86.11 (+12.77%)	84.35 (+16.17%)	84.36 (+18.95%)	(+21.64%)
+ No State $\pi(p_t \mid \cdot)$	62.13 (+24.83%)	87.94 (+14.60%)	87.09 (+12.91%)	80.51 (+15.10%)	(+16.86%)

3.3 分析：难度/稀有度、训练动态、技能发现质量与定性示例

跨稀有度与难度层级的技能学习。 表 2 表明，SKILL-TREE 环境中基于技能的学习可以提升学生模型的总体表现。两个核心问题是：（1）这些技能有多可解释；（2）学习与题目平均难度、技能频率等特征如何相关。图 4 中，我们绘制 Gemma-2B 学生在 DATAENVGYM 的 SKILL-TREE 环境中针对 MATH 训练后的准确率提升；大多数技能都有提升，程度不一。图 4(a) 按平均题目难度（MATH 提供 1—5 分）绘制提升：训练在中等难度区域（约 3.5）提升最大；两端提升较小，微积分与优化（高难度）接近 0 差异，三角学（低难度）甚至下降。图 4(b) 比较训练数据中的技能稀有度（频率的倒数）：稀有技能受益较少，更频繁（即不那么稀有）的技能通常受益更多。综合来看，图 4 表明难度与频率存在一个“甜点区”。低端可能是因为饱和：简单技能从训练中受益较少，因为模型已经表现良好或已饱和。另一端，困难技能或非常稀有的技能可能在教师的训练数据中代表性不足，或更难生成题目，导致学习效果较差；也可能生成的题目对学生太难。在中等难度区间，教师生成有帮助的样例，学生得以学习。类似理论也曾用于人类学习，例如 Vygotsky（1934）的“最近发展区”：学习在略难于学生独立可解、但又并非难到毫无希望的问题上最有效。

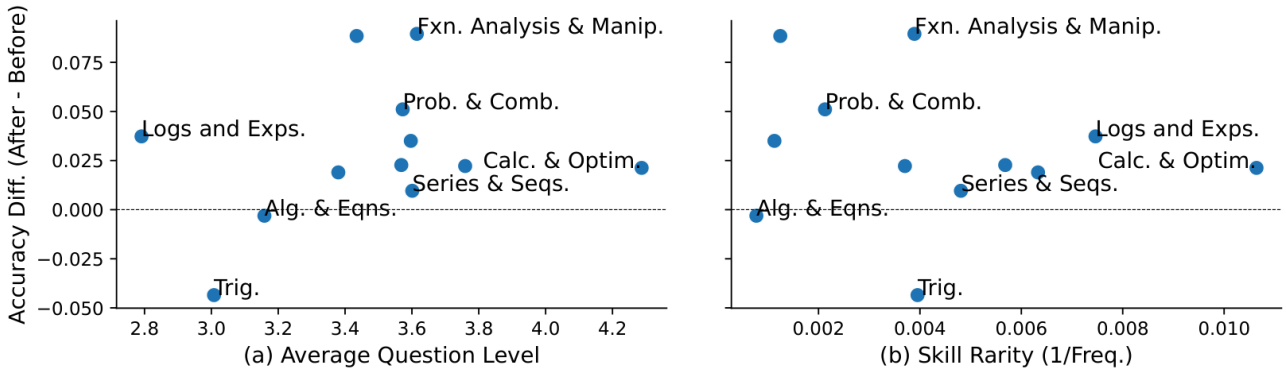


图 4：Gemma-2B 学生在 SKILL-TREE 环境中针对 MATH 训练后，各技能的准确率提升，作为 (a) 题目难度与 (b) 训练数据中技能稀有度（频率倒数）的函数。最大提升出现在中等难度区间与较低稀有度（即更频繁技能）区间。

迭代训练动态。 我们绘制每个任务、每个环境下，整个 DATAENVGYM 运行过程中学生在验证集上的表现变化。每个实验在表现饱和（连续多轮不再提升）后截断。我们使用各环境的“With State”基线智能体，模型与表 2 相同。结果显示学生通常随迭代提升；换言之，基线智能体确实每轮都发现了进一步改进学生的数据

点。此外，三个环境中的智能体行为随任务而不同：在 GQA 上，SKILL-TREE 比其他环境能改进更多轮；在 MATH 上达到平台期；在 LiveCodeBench 上两轮数据生成后即被截断。

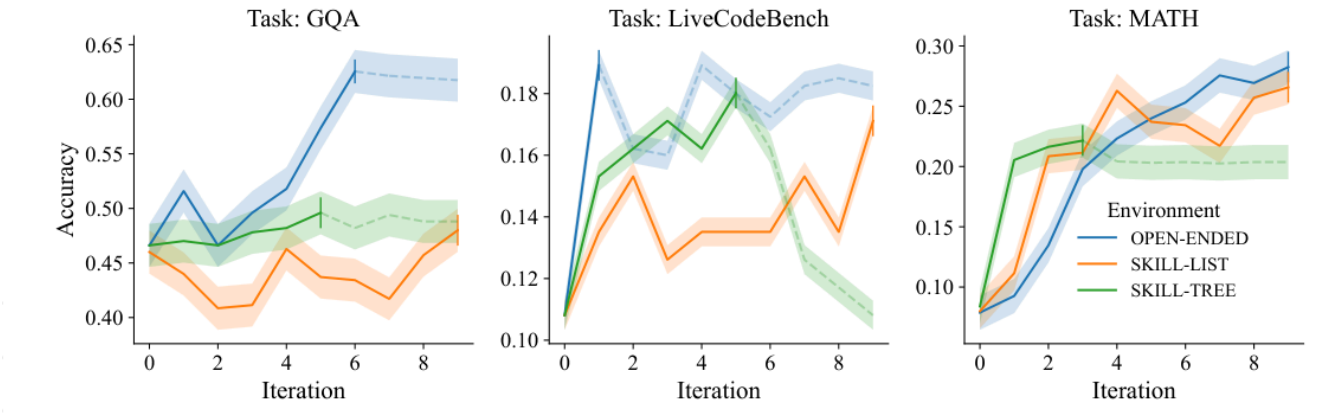


图 5：三个任务上的训练动态。每条线分为实线段（终止于竖线，表示学生达到的最大表现）与虚线（表示超过该点后继续训练的效果）。

技能发现质量的影响。 表 5 展示使用不同技能发现模块的 SKILL-LIST 环境所生成数据训练学生的结果。对每个领域，我们确定一组“oracle”技能。对 GQA，oracle 技能是人工标注技能。由于 MATH 没有人工标注技能，我们通过在测试数据上运行技能发现模块来近似 oracle 技能，从而利用特权信息创建技能。两种设置下，oracle 技能都让教师产生更好的数据并提升学生表现。GQA 上 oracle 技能的提升高于 MATH，可能是因为 MATH 技能仍依赖与推断技能相同的技能发现模块。这是一个有希望的信号：它表明改进技能发现模块可以获得进一步的性能提升。这些结果也凸显了 DATAENVGYM 在替换不同组件方面的实用性：通过为数据生成智能体建立模块化框架，未来工作可以以学生表现为指标，测试框架中的模块与组件。

表 5：DATAENVGYM 允许测试环境组件的多种实现。 这里比较 GQA 与 MATH 的 oracle 技能与推断技能。更好的技能带来更好的教学，从而改进学生。

技能类型	GQA（PaliGemma 3B）	MATH（Gemma2 2B）
教学前	44.18	15.78
SKILL-LIST + Oracle 技能	53.02 (+8.84%)	19.52 (+3.74%)
SKILL-LIST + 推断技能	48.18 (+4.00%)	18.25 (+2.47%)

测试不同的教师策略。 表 6 中，我们把驱动教师智能体的 LLM 策略替换为 GPT-4o-mini，测试不同教师 LLM 策略，尤其是更小、更弱的模型。使用较弱 LLM 时，教师智能体生成的数据对学生改进效果较差。较弱教师在技能结构化环境中更成功（+1.81% 与 +1.75%），而在 OPEN-ENDED 环境中仅 +0.77%。

表 6：教师所产生数据的质量很重要。 这里把教师智能体内部的 GPT-4o 替换为 GPT-4o-mini。

设置/环境	GQA（PaliGemma 3B）	MATH（Gemma2 2B）	LiveCodeBench（Llama3 8B）	平均提升
教学前	44.18	15.78	16.50	-
OPEN-ENDED + GPT-4o-mini	45.51 (+1.33%)	15.78 (+0.00%)	17.50 (+1.00%)	(+0.77%)

设置/环境	GQA（PaliGemma 3B）	MATH（Gemma2 2B）	LiveCodeBench（Llama3 8B）	平均提升
OPEN-ENDED + GPT-4o	47.90 (+3.72%)	23.44 (+7.66%)	18.91 (+2.41%)	(+4.60%)
SKILL-LIST + GPT-4o-mini	46.74 (+2.56%)	16.92 (+1.14%)	17.25 (+1.25%)	(+1.75%)
SKILL-LIST + GPT-4o	48.18 (+4.00%)	19.48 (+3.70%)	18.25 (+1.75%)	(+3.15%)
SKILL-TREE + GPT-4o-mini	48.83 (+4.65%)	16.06 (+0.28%)	17.00 (+0.50%)	(+1.81%)
SKILL-TREE + GPT-4o	49.76 (+5.58%)	18.90 (+3.12%)	17.75 (+1.25%)	(+3.32%)

生成的技能树与样例。 附录 C 的图 13、图 14、图 15 分别展示为 GQA、MATH、LiveCodeBench 发现的技能与子技能的定性示例。对每个子技能，我们还展示教师模型生成的示例数据点。注意，这些数据点是完整生成的，包括图 13 中的图像。每个任务的训练数据结构不同，但生成的训练数据总体符合目标任务在多个技能上所需的风格。

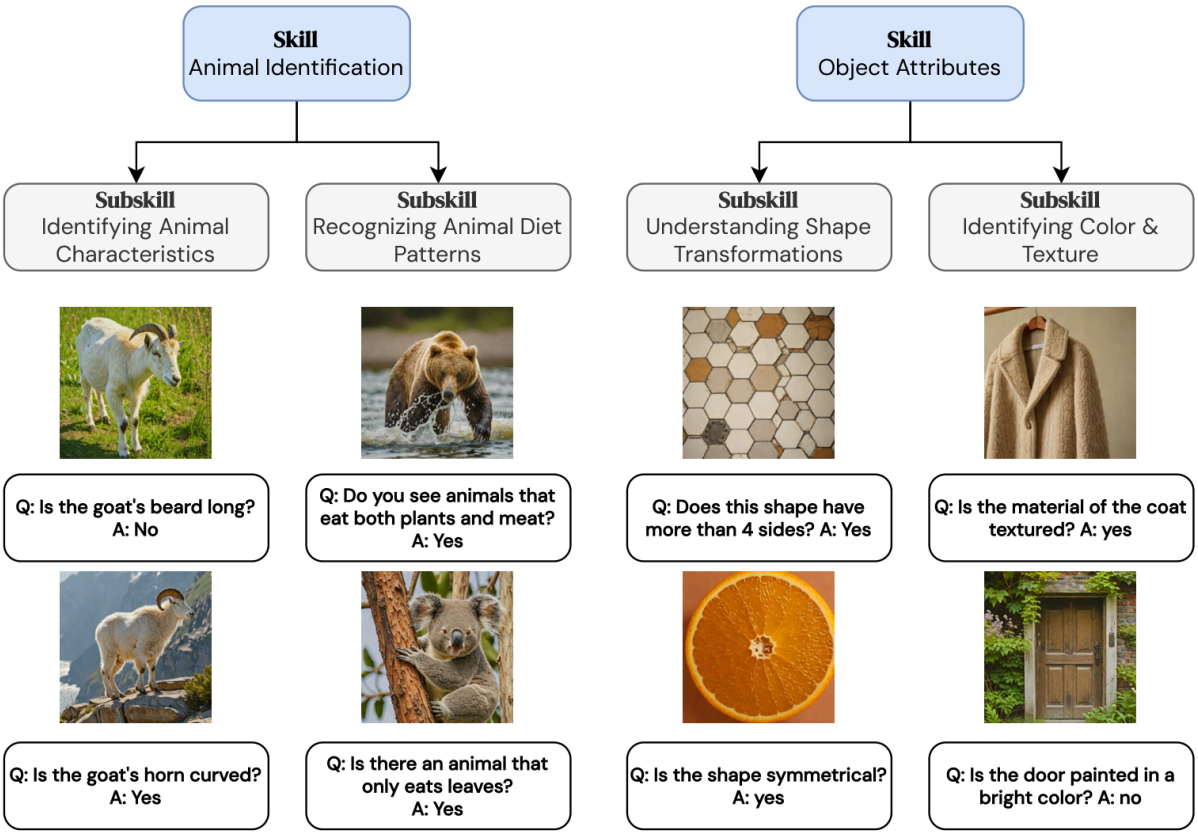


图 13：GQA 技能森林的部分视图。图中展示 13 个已发现技能中的 2 个。对技能树中的每个技能，展示 2 个子技能及该子技能的 2 个生成数据示例。注意所有图像均为生成结果；由于 GQA 中大多数问题是“是/否”问题，答案多为布尔值。

Figure 13: A partial view of a skill forest for GQA. Depicted are 2 out of 15 discovered skills. For each skill in the skill tree, we show 2 subskills and 2 examples of generated data for that subskill. Note that all images are generated and tend to have boolean answers because a majority of the questions in GQA are yes / no questions.

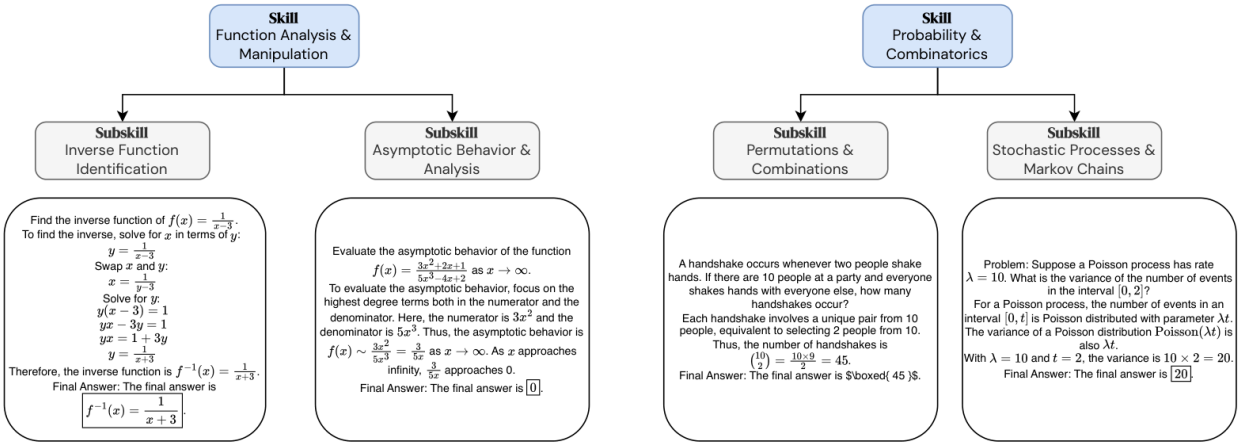


图 14：MATH 技能森林的部分视图。图中展示 12 个已发现技能中的 2 个。对技能树中的每个技能，展示 2 个子技能及该子技能的 1 个生成数据示例。

训练前后的模型预测。 图 17 展示在生成数据上训练如何改变 PaliGemma-3B 学生回答的定性示例。左侧示例属于技能发现中识别出的“Material Identification and Comparison”（材料识别与比较）技能；在生成数据上训练后，模型正确识别材料为“plastic”。这可能是合成数据中椅子可能材料的去偏结果。右侧是另一个棘手示例：

初始答案可能是模型把前景（长颈鹿，“brown and tall”为真）与背景混淆所致；在合成数据上训练后，该错误得到纠正。

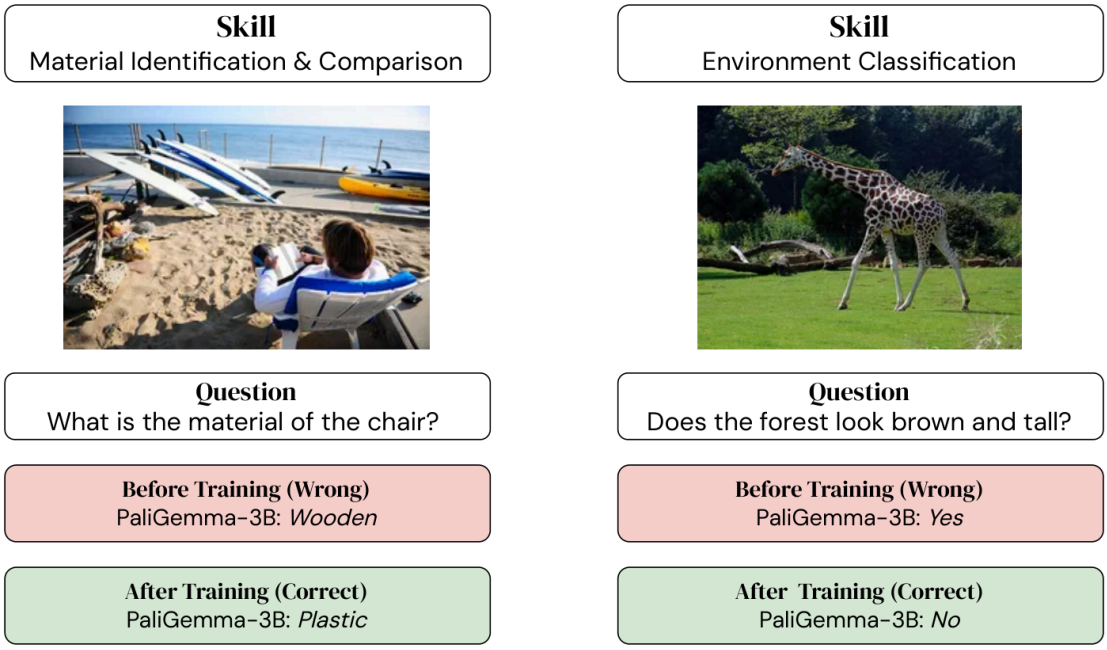


图 17：在生成数据上训练如何改变 PaliGemma-3B 学生回答的定性示例。

4 相关工作

训练环境生成。 在智能体学习框架中，设计训练环境通常成为瓶颈，因为它需要复杂的人工投入（Park et al., 2024）。无监督环境设计（UED）（Dennis et al., 2020; Jiang et al., 2021; Parker-Holder et al., 2022）依据智能体在简单游戏中的得分，逐步提高环境难度；Ruiz et al. (2019) 控制合成数据的分布，以最大化在其上训练模型的准确率。Liesen et al. (2024) 引入元学习方法，为连续控制创建学习环境。在视觉—语言导航（VLN）中，过往工作（Li et al., 2022; Li & Bansal, 2024; Wang et al., 2023c）提出用图像生成模型增强训练环境的视觉多样性。生成也被应用于游戏环境：Cobbe et al. (2020) 生成多样游戏环境以训练 RL 智能体并衡量其泛化。在我们此前的工作 EnvGen（Zala et al., 2024）中，我们持续自适应游戏实例以训练 RL 智能体，用 LLM 依据智能体中间进展的反馈生成不同游戏引擎参数，教授核心技能，从而提升智能体最终表现与学习效率。Yang et al. (2024) 依据用户指定提示生成 3D 具身环境，生成不同风格的房间。Sudhakaran et al. (2023) 与 Todd et al. (2023) 用 LLM 为瓦片式游戏开放式生成关卡。OMNI（Zhang et al., 2023）用 LLM 在开放式环境中做任务选择，OMNI-Epic（Faldor et al., 2024）为 OMNI 增加在模拟机器人环境中生成新任务的能力。总体而言，过往环境生成工作聚焦于简单游戏等受限环境，通常只有少量预定义动作与技能；且多关注发展学生，而非改进数据生成过程本身。相反，本文聚焦数据生成智能体，为教师建立测试床，并把学生视为环境的一部分。此外，我们的工作引入了在数学、视觉问答、编程等多样开放式、更真实设置中带自动技能发现的数据生成环境。

从生成数据中学习。 DATAENVGYM 涉及依据学生模型反馈，以有效方式把任务知识从教师智能体迁移到学生模型。过往的模型间知识迁移研究以知识蒸馏为中心，用大模型输出训练小模型（Hinton et al., 2015;

Buciluǎ et al., 2006; Chen et al., 2020）；与 DATAENVGYM 的过程不同，这一过程通常不是自适应的，依赖固定输入数据集，由较大教师模型处理后训练学生。在 LLM 语境下，符号蒸馏（West et al., 2022）日益常见：从大模型生成文本，用于训练较小模型，例如指令微调（Wang et al., 2023a），或把大型专有模型的思维链推理（Wei et al., 2022）蒸馏到小模型（Magister et al., 2023; Shridhar et al., 2023; Fu et al., 2023; Ho et al., 2023; Saha et al., 2023; Mukherjee et al., 2023; Mitra et al., 2023; Chen et al., 2024）。DATAENVGYM 环境中数据生成智能体所执行的教学，与知识蒸馏设置的不同在于：模型输入本身由模型生成（而非取自现有数据集）；且输入依据学生模型的反馈与错误动态确定，而知识蒸馏中由固定数据集决定或一次性生成。注意，DATAENVGYM 兼容不同的学生训练方法（如符号蒸馏、基于 logit 的损失等），这些都可以替换进我们的模块化环境。

5 结论

我们提出 DATAENVGYM，一个由教师环境构成的测试床，用于开发模块化数据生成智能体（即教师）与环境。在 DATAENVGYM 中，教师智能体为学生模型生成训练数据，并从环境接收学生表现反馈。我们提供三种结构化程度不同的环境实例。在四个多样领域（视觉问答、数学、编程、工具使用）上，我们证明所引入的示例教师能在所有 DATAENVGYM 环境中改进学生。我们分析了 DATAENVGYM，包括训练动态、不同难度层级的表现以及技能发现模块质量的影响。我们希望本工作能通过提供评估与改进数据生成智能体、引擎与反馈机制的测试床，促进这些方向的未来进展。

致谢

本工作得到 DARPA ECOLE 计划（No. HR00112390060）、NSF-AI Engage Institute（DRL-2112635）、DARPA Machine Commonsense (MCS) 资助（N66001-19-2-4031）、ARO 奖项 W911NF2110220、ONR 资助 N00014-23-1-2356、Microsoft Accelerate Foundation Models Research (AFMR) 资助计划以及 Bloomberg Data Science PhD Fellowship 的支持。本文观点属于作者，不代表资助机构。

参考文献

Marc Bellemare, Sriram Srinivasan, Georg Ostrovski, Tom Schaul, David Saxton, and Remi Munos. Unifying count-based exploration and intrinsic motivation. In Advances in neural information processing systems, volume 29, 2016.

Lucas Beyer, Andreas Steiner, André Susano Pinto, Alexander Kolesnikov, Xiao Wang, Daniel Salz, Maxim Neumann, Ibrahim Alabdulmohsin, Michael Tschannen, Emanuele Bugliarello, Thomas Unterthiner, Daniel Keysers, Skanda Koppula, Fangyu Liu, Adam Grycner, Alexey Gritsenko, Neil Houlsby, Manoj Kumar, Keran Rong, Julian Eisenschlos, Rishabh Kabra, Matthias Bauer, Matko Bošnjak, Xi Chen, Matthias Minderer, Paul Voigtlaender, Ioana Bica, Ivana Balazevic, Joan Puigcerver, Pinelopi Papalampidi, Olivier Henaff, Xi Xiong, Radu Soricut, Jeremiah Harmsen, and Xiaohua Zhai. Paligemma: A versatile 3b vlm for transfer, 2024. URL https://arxiv.org/abs/2407.07726.

Cristian Bucilu ˇ a, Rich Caruana, and Alexandru Niculescu-Mizil. Model compression. In Proceedings of the 12th ACM SIGKDD international conference on Knowledge discovery and data mining, pp. 535–541, 2006. URL https://dl.acm.org/doi/abs/10.1145/1150402.1150464.

Yuri Burda, Harrison Edwards, Amos Storkey, and Oleg Klimov. Exploration by random network distillation. In International Conference on Learning Representations, 2019. URL https: //openreview.net/forum?id=H1lJJnR5Ym.

Justin Chen, Swarnadeep Saha, Elias Stengel-Eskin, and Mohit Bansal. Magdi: Structured distillation of multi-agent interaction graphs improves reasoning in smaller language models. In Forty-first International Conference on Machine Learning, 2024.

Ting Chen, Simon Kornblith, Kevin Swersky, Mohammad Norouzi, and Geoffrey E Hinton. Big self-supervised models are strong semi-supervised learners. Advances in neural information processing systems, 33:22243–22255, 2020. URL https://arxiv.org/abs/2006.10029. Yew Ken Chia, Pengfei

Hong, Lidong Bing, and Soujanya Poria. Instructeval: Towards holistic evaluation of instruction-tuned large language models. In Proceedings of the First edition of the Workshop on the Scaling Behavior of Large Language Models (SCALE-LLM 2024), pp. 35–64, 2024.

Karl Cobbe, Chris Hesse, Jacob Hilton, and John Schulman. Leveraging procedural generation to benchmark reinforcement learning. In International conference on machine learning, pp. 2048–2056. PMLR, 2020.

Michael Dennis, Natasha Jaques, Eugene Vinitsky, Alexandre Bayen, Stuart Russell, Andrew Critch, and Sergey Levine. Emergent complexity and zero-shot transfer via unsupervised environment design. Advances in neural information processing systems, 33:13049–13061, 2020.

Aniket Didolkar, Anirudh Goyal, Nan Rosemary Ke, Siyuan Guo, Michal Valko, Timothy Lillicrap, Danilo Rezende, Yoshua Bengio, Michael Mozer, and Sanjeev Arora. Metacognitive Capabilities of LLMs: An Exploration in Mathematical Problem Solving, May 2024.

Ning Ding, Yulin Chen, Bokai Xu, Yujia Qin, Shengding Hu, Zhiyuan Liu, Maosong Sun, and Bowen Zhou. Enhancing chat language models by scaling high-quality instructional conversations. In Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing, pp. 3029–3051, 2023.

Maxence Faldor, Jenny Zhang, Antoine Cully, and Jeff Clune. Omni-epic: Open-endedness via models of human notions of interestingness with environments programmed in code, 2024.

Haotian Fu, Pratyusha Sharma, Elias Stengel-Eskin, George Konidaris, Nicolas Le Roux, Marc-Alexandre Côté, and Xingdi Yuan. Language-guided skill learning with temporal variational inference. In Forty-first International Conference on Machine Learning, 2024.

Yao Fu, Hao Peng, Litu Ou, Ashish Sabharwal, and Tushar Khot. Specializing smaller language models towards multi-step reasoning. In Andreas Krause, Emma Brunskill, Kyunghyun Cho, Bar-bara Engelhardt, Sivan Sabato, and Jonathan Scarlett (eds.), Proceedings of the 40th International Conference on Machine Learning, volume 202 of Proceedings of Machine Learning Research, pp. 10421–10430. PMLR, 23–29 Jul 2023. URL https://proceedings.mlr.press/v202/fu23d.html.

Deep Ganguli, Liane Lovitt, Jackson Kernion, Amanda Askell, Yuntao Bai, Saurav Kadavath, Ben Mann, Ethan Perez, Nicholas Schiefer, Kamal Ndousse, et al. Red teaming language models to reduce harms: Methods, scaling behaviors, and lessons learned. arXiv preprint arXiv:2209.07858, 2022.

Tong Gao, Shivang Singh, and Raymond Mooney. Towards automated error analysis: Learning to characterize errors. In The International FLAIRS Conference Proceedings, volume 35, 2022. Gemma Team. Gemma 2: Improving open language models at a practical size, 2024. URL https://arxiv.org/abs/2408.00118.

Karol Gregor, Danilo Jimenez Rezende, and Daan Wierstra. Variational intrinsic control. arXiv preprint arXiv:1611.07507, 2016. Tanmay Gupta and Aniruddha Kembhavi. Visual programming: Compositional visual reasoning without training. In Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition, pp. 14953–14962, 2023.

Dan Hendrycks, Collin Burns, Saurav Kadavath, Akul Arora, Steven Basart, Eric Tang, Dawn Song, and Jacob Steinhardt. Measuring mathematical problem solving with the math dataset. In Thirty-fifth Conference on Neural Information Processing Systems Datasets and Benchmarks Track (Round 2), 2021.

Geoffrey Hinton, Oriol Vinyals, and Jeff Dean. Distilling the knowledge in a neural network. arXiv preprint arXiv:1503.02531, 2015. URL https://arxiv.org/abs/1503.02531.

Namgyu Ho, Laura Schmid, and Se-Young Yun. Large language models are reasoning teachers. In Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), pp. 14852–14882, Toronto, Canada, July 2023. Association for Com-putational Linguistics. doi:10.18653/v1/2023.acl-long.830. URL https://aclanthology. org/2023.acl-long.830. Edward J Hu, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, Weizhu Chen, et al. Lora: Low-rank adaptation of large language models. In International Conference on Learning Representations, 2022. Drew A Hudson and Christopher D Manning. Gqa: A new dataset for real-world visual reasoning and compositional question answering. In Proceedings of the IEEE/CVF conference on computer vision and pattern recognition, pp. 6700–6709, 2019.

Srinivasan Iyer, Xi Victoria Lin, Ramakanth Pasunuru, Todor Mihaylov, Daniel Simig, Ping Yu, Kurt Shuster, Tianlu Wang, Qing Liu, Punit Singh Koura, et al. Opt-iml: Scaling language model instruction meta learning through the lens of generalization. arXiv preprint arXiv:2212.12017, 2022.

Naman Jain, King Han, Alex Gu, Wen-Ding Li, Fanjia Yan, Tianjun Zhang, Sida Wang, Armando Solar-Lezama, Koushik Sen, and Ion Stoica. Livecodebench: Holistic and contamination free evaluation of large language models for code. arXiv preprint arXiv:2403.07974, 2024.

Minqi Jiang, Michael Dennis, Jack Parker-Holder, Jakob Foerster, Edward Grefenstette, and Tim Rocktäschel. Replay-guided adversarial environment design. Advances in Neural Information Processing Systems, 34:1884–1897, 2021.

Baiqi Li, Zhiqiu Lin, Wenxuan Peng, Jean de Dieu Nyandwi, Daniel Jiang, Zixian Ma, Simran Khanuja, Ranjay Krishna, Graham Neubig, and Deva Ramanan. Naturalbench: Evaluating vision-language models on natural adversarial samples. In The Thirty-eight Conference on Neural Information Processing Systems Datasets and Benchmarks Track, 2024. Jialu Li and Mohit Bansal. Panogen: Text-conditioned panoramic environment generation for vision-and-language navigation. Advances in Neural Information Processing Systems, 36, 2024.

Jialu Li, Hao Tan, and Mohit Bansal. Envedit: Environment editing for vision-and-language naviga-tion. In Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition, pp. 15407–15417, 2022.

Jarek Liesen, Chris Lu, Andrei Lupu, Jakob N. Foerster, Henning Sprekeler, and Robert T. Lange. Discovering minimal reinforcement learning environments, 2024. URL https://arxiv.org/abs/2406.12589. Llama Team. The llama 3 herd of models, 2024. URL https://arxiv.org/abs/2407. 21783.

Shayne Longpre, Le Hou, Tu Vu, Albert Webson, Hyung Won Chung, Yi Tay, Denny Zhou, Quoc V Le, Barret Zoph, Jason Wei, et al. The flan collection: Designing data and methods for effective instruction tuning. In International Conference on Machine Learning, pp. 22631–22648. PMLR, 2023. Ilya Loshchilov and Frank Hutter. Decoupled weight decay regularization. arXiv preprint arXiv:1711.05101, 2017.

Zixian Ma, Weikai Huang, Jieyu Zhang, Tanmay Gupta, and Ranjay Krishna. m&m's: A benchmark to evaluate tool-use for multi-step multi-modal tasks. EECV 2024, 2024. Lucie Charlotte Magister, Jonathan Mallinson, Jakub Adamek, Eric Malmi, and Aliaksei Severyn. Teaching small language models to reason. In Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 2: Short Papers), pp. 1773–1781, Toronto, Canada, July 2023. Association for Computational Linguistics. doi:10.18653/v1/2023.acl-short.151. URL https://aclanthology.org/2023.acl-short.151.

Arindam Mitra, Luciano Del Corro, Shweti Mahajan, Andres Codas, Clarisse Simoes, Sahaj Agarwal, Xuxi Chen, Anastasia Razdaibiedina, Erik Jones, Kriti Aggarwal, et al. Orca 2: Teaching small language models how to reason. arXiv preprint arXiv:2311.11045, 2023. URL https: //arxiv.org/abs/2311.11045.

Subhabrata Mukherjee, Arindam Mitra, Ganesh Jawahar, Sahaj Agarwal, Hamid Palangi, and Ahmed Awadallah. Orca: Progressive learning from complex explanation traces of gpt-4. arXiv preprint arXiv:2306.02707, 2023. URL https://arxiv.org/abs/2306.02707. OpenAI. Hello gpt-4o, 2024. URL https://openai.com/index/hello-gpt-4o.

Younghyo Park, Gabriel B. Margolis, and Pulkit Agrawal. Automatic Environment Shaping is the Next Frontier in RL, July 2024.

Jack Parker-Holder, Minqi Jiang, Michael Dennis, Mikayel Samvelyan, Jakob Foerster, Edward Grefenstette, and Tim Rocktäschel. Evolving curricula with regret-based environment design. In International Conference on Machine Learning, pp. 17473–17498. PMLR, 2022.

Ethan Perez, Saffron Huang, Francis Song, Trevor Cai, Roman Ring, John Aslanides, Amelia Glaese, Nat McAleese, and Geoffrey Irving. Red teaming language models with language models. arXiv preprint arXiv:2202.03286, 2022.

Seungeun Rho, Laura Smith, Tianyu Li, Sergey Levine, Xue Bin Peng, and Sehoon Ha. Language guided skill discovery. arXiv preprint arXiv:2406.06615, 2024.

Nataniel Ruiz, Samuel Schulter, and Manmohan Chandraker. Learning to simulate. In 7th Interna-tional Conference on Learning Representations, ICLR 2019, New Orleans, LA, USA, May 6-9, 2019. OpenReview.net, 2019. URL https://openreview.net/forum?id=HJgkx2Aqt7.

Swarnadeep Saha, Peter Hase, and Mohit Bansal. Can language models teach weaker agents? teacher explanations improve students via personalization.

In NeurIPS, 2023. URL https: //arxiv.org/abs/2306.09299.

Axel Sauer, Dominik Lorenz, Andreas Blattmann, and Robin Rombach. Adversarial diffusion distillation, 2023. URL https://arxiv.org/abs/2311.17042.

Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Mingchuan Zhang, YK Li, Yu Wu, and Daya Guo. Deepseekmath: Pushing the limits of mathematical reasoning in open language models. arXiv preprint arXiv:2402.03300, 2024.

Pratyusha Sharma, Antonio Torralba, and Jacob Andreas. Skill induction and planning with latent language. In Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), pp. 1713–1726, 2022.

Luísa Shimabucoro, Sebastian Ruder, Julia Kreutzer, Marzieh Fadaee, and Sara Hooker. Llm see, llm do: Guiding data generation to target non-differentiable objectives. arXiv preprint arXiv:2407.01490, 2024.

Kumar Shridhar, Alessandro Stolfo, and Mrinmaya Sachan. Distilling reasoning capabilities into smaller language models. In Findings of the Association for Computational Linguistics: ACL 2023, pp. 7059–7073, 2023. URL https://arxiv.org/abs/2212.00193.

Shyam Sudhakaran, Miguel González Duque, Matthias Freiberger, Claire Glanois, Elias Najarro, and Sebastian Risi. Mariogpt: Open-ended text2level generation through large language models. In Alice Oh, Tristan Naumann, Amir Globerson, Kate Saenko, Moritz Hardt, and Sergey Levine (eds.), Advances in Neural Information Processing Systems 36: Annual Conference on Neural Information Processing Systems 2023, NeurIPS 2023, New Orleans, LA, USA, December 10 - 16, 2023, 2023. URL http://papers.nips.cc/paper_files/paper/2023/hash/a9bbeb2858dfbdbd4c19814e5d80ec60-Abstract-Conference.html.

Rohan Taori, Ishaan Gulrajani, Tianyi Zhang, Yann Dubois, Xuechen Li, Carlos Guestrin, Percy Liang, and Tatsunori B Hashimoto. Stanford alpaca: An instruction-following llama model, 2023.

Graham Todd, Sam Earle, Muhammad Umair Nasir, Michael Cerny Green, and Julian Togelius. Level generation through large language models. In Phil Lopes, Filipe Luz, Antonios Liapis, and Henrik Engström (eds.), Proceedings of the 18th International Conference on the Foundations of Digital Games, FDG 2023, Lisbon, Portugal, April 12-14, 2023, pp. 70:1–70:8. ACM, 2023. doi:10.1145/3582437.3587211. URL https://doi.org/10.1145/3582437.3587211.

Hugo Touvron, Louis Martin, Kevin Stone, Peter Albert, Amjad Almahairi, Yasmine Babaei, Nikolay Bashlykov, Soumya Batra, Prajjwal Bhargava, Shruti Bhosale, et al. Llama 2: Open foundation and fine-tuned chat models. arXiv preprint arXiv:2307.09288, 2023. Lev S Vygotsky. Thought and language. 1934.

Yizhong Wang, Yeganeh Kordi, Swaroop Mishra, Alisa Liu, Noah A. Smith, Daniel Khashabi, and Hannaneh Hajishirzi. Self-instruct: Aligning language models with self-generated instruc-tions. In Anna Rogers, Jordan Boyd-Graber, and Naoaki Okazaki (eds.), Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Pa-pers), pp. 13484–13508, Toronto, Canada, July 2023a. Association for Computational Lin-guistics. doi:10.18653/v1/2023.acl-long.754. URL https://aclanthology.org/2023. acl-long.754.

Zige Wang, Wanjun Zhong, Yufei Wang, Qi Zhu, Fei Mi, Baojun Wang, Lifeng Shang, Xin Jiang, and Qun Liu. Data management for training large language models: A survey. 2023b. URL https://api.semanticscholar.org/CorpusID:265609639.

Zun Wang, Jialu Li, Yicong Hong, Yi Wang, Qi Wu, Mohit Bansal, Stephen Gould, Hao Tan, and Yu Qiao. Scaling data generation in vision-and-language navigation. In Proceedings of the IEEE/CVF International Conference on Computer Vision, pp. 12009–12020, 2023c. Christopher John Cornish Hellaby Watkins. Learning from delayed rewards. 1989.

Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Fei Xia, Ed Chi, Quoc V Le, Denny Zhou, et al. Chain-of-thought prompting elicits reasoning in large language models. Advances in Neural Information Processing Systems, 35:24824–24837, 2022. URL https://arxiv.org/abs/2201.11903.

Peter West, Chandra Bhagavatula, Jack Hessel, Jena Hwang, Liwei Jiang, Ronan Le Bras, Ximing Lu, Sean Welleck, and Yejin Choi. Symbolic knowledge distillation: from general language models to commonsense models. In Marine Carpuat, Marie-Catherine de Marneffe, and Ivan Vladimir Meza Ruiz (eds.), Proceedings of the 2022 Conference of the North American Chapter of the Asso-ciation for Computational Linguistics: Human Language Technologies, pp. 4602–4625, Seattle, United States, July 2022. Association for Computational Linguistics. doi:10.18653/v1/2022.naacl-main.341. URL https://aclanthology.org/2022.naacl-main.341.

Thomas Wolf, Lysandre Debut, Victor Sanh, Julien Chaumond, Clement Delangue, Anthony Moi, Pierric Cistac, Tim Rault, Rémi Louf, Morgan Funtowicz, Joe Davison, Sam Shleifer, Patrick von Platen, Clara Ma, Yacine Jernite, Julien Plu, Canwen Xu, Teven Le Scao, Sylvain Gugger, Mariama Drame, Quentin Lhoest, and Alexander M. Rush. Transformers: State-of-the-art natural language processing. In Proceedings of the 2020 Conference on Empirical Methods in Natural Language Pro-cessing: System Demonstrations, pp. 38–45, Online, October 2020. Association for Computational Linguistics. URL https://www.aclweb.org/anthology/2020.emnlp-demos.6.

Tongshuang Wu, Marco Tulio Ribeiro, Jeffrey Heer, and Daniel S Weld.

Errudite: Scalable, repro-ducible, and testable error analysis. In Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics, pp. 747–763, 2019.

Xiyang Wu, Tianrui Guan, Dianqi Li, Shuaiyi Huang, Xiaoyu Liu, Xijun Wang, Ruiqi Xian, Ab-hinav Shrivastava, Furong Huang, Jordan L. Boyd-Graber, Tianyi Zhou, and Dinesh Manocha. Autohallusion: Automatic generation of hallucination benchmarks for vision-language models. ArXiv, abs/2406.10900, 2024. URL https://api.semanticscholar.org/CorpusID: 270560551.

Yue Yang, Fan-Yun Sun, Luca Weihs, Eli VanderBilt, Alvaro Herrasti, Winson Han, Jiajun Wu, Nick Haber, Ranjay Krishna, Lingjie Liu, et al. Holodeck: Language guided generation of 3d embodied ai environments. In Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition, pp. 16227–16237, 2024.

Abhay Zala, Jaemin Cho, Han Lin, Jaehong Yoon, and Mohit Bansal. Envgen: Generating and adapting environments via llms for training embodied agents.

In COLM, 2024.

Jenny Zhang, Joel Lehman, Kenneth Stanley, and Jeff Clune. Omni: Open-endedness via models of human notions of interestingness. arXiv preprint arXiv:2306.01711, 2023.

Jieyu Zhang, Weikai Huang, Zixian Ma, Oscar Michel, Dong He, Tanmay Gupta, Wei-Chiu Ma, Ali Farhadi, Aniruddha Kembhavi, and Ranjay Krishna. Task me anything. arXiv preprint arXiv:2406.11775, 2024.

Yaowei Zheng, Richong Zhang, Junhao Zhang, Yanhan Ye, Zheyan Luo, Zhangchi Feng, and Yongqiang Ma. Llamafactory: Unified efficient fine-tuning of 100+ language models. In Pro-ceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 3: System Demonstrations), Bangkok, Thailand, 2024. Association for Computational Linguistics. URL http://arxiv.org/abs/2403.13372.

Chunting Zhou, Pengfei Liu, Puxin Xu, Srinivasan Iyer, Jiao Sun, Yuning Mao, Xuezhe Ma, Avia Efrat, Ping Yu, Lili Yu, et al. LIMA: Less is more for alignment. Advances in Neural Information Processing Systems, 36, 2024.

附录

附录包含额外相关工作（附录 A）、额外方法细节（附录 B）、定性示例（附录 C）、生成数据上的准确率实验（附录 D），以及关于数据与训练时间相对重要性的消融（附录 E）。

A 额外相关工作

技能发现。 强化学习中有一类工作研究无监督技能发现：智能体在无显式奖励下学习多样的涌现行为。这类工作大多通过最大化智能体轨迹多样性来帮助发现新技能，例如探索鼓励奖励（Gregor et al., 2016; Bellemare et al., 2016）以及在动作采样中加入随机性（Watkins, 1989; Burda et al., 2019）。然而，这类方法需要很长的探索步数，若智能体动作成本不可忽略则代价高昂。近期工作还提出利用预训练语言模型中的知识辅助技能发现（Sharma et al., 2022; Rho et al., 2024; Fu et al., 2024）并采样新技能（Didolkar et al., 2024）。技能发现的发展与改进与 DATAENVGYM 互补：在 SKILL-LIST 与 SKILL-TREE 环境（第 2.1.2 节）中，技能发现用于从数据中动态提取人类可解释的技能并创建学生反馈。这些技能帮助识别模型弱点，并为数据生成过程提供条件；我们

发现改进技能发现可以提升学生模型表现（参见表 5），这指明了未来工作方向。此外，通过在其环境中使用技能发现，DATAENVGYM 不仅帮助开发与测试可解释智能体，也为技能发现提供了具体的外部评估，使相互竞争的技能发现方法能够依据其改进下游智能体表现的程度被评估与比较。

模型弱点发现。 在不同场景中测试已训练的机器学习模型，对缓解意外故障至关重要，开发者可据此主动预防此类行为并在薄弱技能上微调模型。传统上，模型弱点通过雇佣人工标注者，要求他们在不同场景中构造对抗输入，并检查模型输出何时错误或不合预期来识别（Ganguli et al., 2022）。近期工作探索通过在不同场景中创建测试用例来自动发现模型弱点，流水线可由传统 ML 模型（Gao et al., 2022; Wu et al., 2019）或 LLM（Perez et al., 2022; Zhang et al., 2024; Wu et al., 2024）组成。这些方法往往聚焦越狱或红队等对抗场景，使模型对越来越强的对手更鲁棒。我们的框架是正交的：DATAENVGYM 不聚焦防御对手，而是让学生与教师（数据生成智能体）合作，共同提升学生表现。

B 额外方法细节

图 6 给出核心环境循环的更详细走查。图 6 的示例描绘 OPEN-ENDED 环境下多模态领域、以 GQA 为目标数据集的一次经验。该循环高度模块化，可通过更换评估数据集来面向不同的数据分布；数据生成策略与数据生成引擎同样可以轻松修改。

B.1 训练细节

B.1.1 GQA

我们使用 Transformers（Wolf et al., 2020）库训练。PaliGemma-3b-pt-224（Beyer et al., 2024）训练 10 个 epoch，使用 AdamW（Loshchilov & Hutter, 2017）优化器，2 步 warmup，学习率 2×10^{-5}，权重衰减 10^{-6}，BF16 数据类型，批大小 16。我们应用 LoRA（Hu et al., 2022），秩 16、alpha 32、无 bias、dropout 0.05，作用于所有线性层。

B.1.2 LiveCodeBench 与 MATH

我们使用 Transformers（Wolf et al., 2020）与 Llama-Factory（Zheng et al., 2024）库训练。所有数据格式化为 Alpaca（Taori et al., 2023）的指令—回答对。使用 Adam 优化器，批大小 16，余弦学习率调度，warmup 比例 0.1，训练 3 个 epoch，FP16 数据类型。LoRA 作用于所有线性层，秩 16、alpha 32、无 bias、dropout 0.05。所有训练样例截断到最多 1024 个 token。

B.2 LLM 细节

LLM 配置。 以下模块使用 gpt-4o-2024-08-06（OpenAI, 2024）：技能发现（第 2.1.2 节；SKILL-LIST 环境）、数据生成策略（第 2.2.1 节，SKILL-LIST 环境）、数据生成引擎（第 2.2.2 节；OPEN-ENDED、SKILL-LIST、SKILL-TREE 环境）。温度 0，top-p 1.0，即默认 API 设置。我们使用 Instructor 库从 LLM 调用产生结构化输出，并自动把 LLM 调用解析为 Python 对象。

提示词模板。 我们提供技能发现（图 7）、GQA（图 8）/MATH（图 9）/LiveCodeBench（图 10）数据生成，以及 OPEN-ENDED（图 11）与 SKILL-LIST（图 12）环境数据生成策略的 LLM 提示词模板。

> 以下为附录中的提示词模板原文（英文）。这些模板是可直接运行的操作性文本，变量名、Jinja 语法与代码结构保持原样，仅在此保留以便复现；正文翻译不改变其内容。

图 7：技能发现模块提示词模板（SKILL-LIST/SKILL-TREE）

```
Published as a conference paper at ICLR 2025
QUESTION_TO_SKILL_TEMPLATE = jinja2.Template(
    """Consider this question about {{ topic }}.
The question tests the {{ topic }} skills of a {{ student_description }}.
Label this question with a specific skill that would be required to solve the question.
You should be able to use the skill as a dictionary key in python.
The skill name should be lower case letters only.
The skill name should be very descriptive and you may use multiple words to describe the skills required in the
question.
If you do use multiple words per question, then join them by an underscore.
Question: {{question}}
"""
)
SKILL_TO_CATEGORIES_TEMPLATE = jinja2.Template(
    """Here is a list of skills required by a {{ student_description }} to solve questions about {{ topic }}:
{% for skill in skills %}
- {{- skill -}}
{% endfor %}
Reduce the number of unique skills by grouping similar skills into categories and give a descriptive name to each
category.
When choosing categories, consider the following:
- The categories should be mutually exclusive.
- The categories should be collectively exhaustive.
- The categories should be descriptive of the skills they contain.
When designing the skill categories, keep in mind that we want to use the skill categories to guide training data
collection to improve a {{ student_descriptor
}}'s performance on {{ topic }} tasks.
Design the skill categories so that collecting data for each category will help improve the model's performance
on the underlying skills.
{% if num_categories is not none %}
Group the skills into at least {{ num_categories }} categories.
{% endif %}
{% if no_function_calling %}
Respond with a list of lines, where each line is a category name followed by a colon and a list of representative
skills for that category.
Here is a Python template demonstrating the expected format:
```python3
template = "{index}. {skill_category}: {representative_skills}
```
Here are examples:
1. Bird Identification: identifying_birds, recognizing_birds, bird_species
2. Welding: welding_steel, welding_aluminum, welding_titanium
A skill category may encompass as many skill as you think are appropriate, but only list up to 3 representative
skills.
Produce plain text, do not wrap the your response in backticks or triple backticks.
Write nothing but the lines that follow the template.
{% endif %}
"""
)
LABEL_QUESTIONS_WITH_SKILL_CATEGORIES_PROMPT = jinja2.Template(
    """Your task is to identify the skill required to solve a question that tests {{ topic }}.
Here is a list of possible skills required by the question:
{% for skill in skills %}
- {{- skill -}}
{% endfor %}
Label the question with one skill from the list, and provide a reason for your choice.
You must ALWAYS choose a skill from the list of skills provided.
Question: {{question}}
"""
)
```

图 8：GQA 数据生成提示词模板

```
Published as a conference paper at ICLR 2025
You are a experienced machine learning engineer and your role is create training data for a model.
Here are some examples of the style of question the model will be answering, and the correct response to the question:
{% for example in examples %}
- instruction: {{ example.instruction }}
- response: {{ example.ground_truth_label }}
{% endfor %}
We will focus on improving skills underneath the category of "{{ subskill }}".
You will propose hypotheses about what training data the model needs to improve its skills under "{{ subskill }}".
The hypotheses will contain specifications of the training data, and we will generate the data from those specifications, and then train the model on the data.
The training data you produce must be valid JSON using the provided schema.
Here are descriptions of the fields in the schema:
- "instruction": The instruction you want the model to respond to.
- "image": The description of an image the instruction is about.
- "response": The correct response to the instruction.
When crafting the training data, consider the following:
- the instructions should be similar in style, length, and complexity to the examples provided
- the images should be relevant to the instruction
- the responses should be similar in style, length, and complexity to the examples provided
- think about what knowledge the model might be missing that would help it answer the question correctly, and craft your training data to give it that knowledge
- each response should be a logically _correct_ response to the instruction in the context of the image description
- the training data should be diverse and help the model improve on "{{ subskill }}"
Produce {{ num_data_specs }} specifications for training data.
```

图 9：MATH 数据生成提示词模板

```
You are an experienced math educator and your task is to create math problems for improving a student's skills in solving math problems.
{% if already_generated_data %}
Here are some problems that you have already written:
{% for data in already_generated_data %}
- Problem: {{ data.problem }}
  - Chain of Thought: {{ data.chain_of_thought }}
  - Final Answer: {{ data.final_answer }}
{% endfor %}
{% endif %}
Each problem should improve the student's ability to solve problems under the category of "{{ subskill }}".
Each problem should require the student to know the concept of "{{ subskill }}".
The problems you produce must be valid JSON using the provided schema.
Here are descriptions of the fields in the schema:
- "problem": The math problem you want the model to solve. Ensure this is valid LaTeX that is properly escaped for representation as a string in Python.
- "chain_of_thought": A step-by-step explanation of how to solve the problem. Ensure this is valid LaTeX that is properly escaped for representation as a string in
Python.
- "final_answer": The final answer to the problem as a LaTeX string. For example '17' or '\\frac{1}{2} or `\\matrix{1 & 2 \\cr 3 & 4}`. Do not write a sentence here, just
the answer.
Propose {{ data_budget }} new problems.
```

图 10：LiveCodeBench 数据生成提示词模板

```
Published as a conference paper at ICLR 2025
You are an expert Python engineer and competitive programming tutor.
You are helping a junior engineer improve their coding skills.
{% if lcb_examples %}
Here are representative examples of the kind of coding problems the junior engineer is facing.
{% for example in lcb_examples %}
Problem:
```

````
{{ example.instruction }}
{% if example.starter_code %}
Starter Code:
{{ example.starter_code }}
{% endif %}
{% if example.solution %}
Solution:
{{ example.solution }}
{% endif %}
{% endfor %}
{% endif %}
You are focusing on problems requiring skills in the category of "{{ subskill }}".
You will propose a new set of problems that require applying skills in the category of "{{ subskill }}".
The problems you propose should be such that solving them will help the junior engineer improve their skills in
the category of "{{ subskill }}".
Here are some guidelines:
- the problems should be similar to coding problems on platforms like LeetCode, Codeforces, etc.
- the problems should require applying skills in "{{ subskill }}"
- only propose problems that YOU KNOW the solution to. This is CRITICAL.
# Output Format
For each problem, you need to include the following:
- instruction: A complete problem statement that would be found in a place like LeetCode. This will be shown
verbatim to the junior engineer.
    - This should include an example input / output and a concise explanation for why the output is correct.
    - Do not write "### Question", just output the problem statement.
- starter_code: The starter code to the problem. Not all problems need starter code.
If you are including starter code, it should be formatted as follows:
```python
class Solution:
    def functionWithMeaningfulName(self, parameter_1: list[SomeType], parameter_2: AnotherType):
        # YOUR CODE HERE
```
Keep "# YOUR CODE HERE" in the code block so the junior engineer knows where to fill in the solution.
You can change functionWithMeaningfulName to anything you want.
Don't forget to also change the parameter names to something that makes sense for the problem.
Propose no more than {{ num_data_specs }} new problems.
{{ num_no_starter_code_problems }} should have no starter code.
The remaining {{ num_data_specs - num_no_starter_code_problems }} should have starter code.
````

图 11：OPEN-ENDED 环境数据生成策略提示词模板

```
Published as a conference paper at ICLR 2025
You are a experienced engineer and your role is to provide training data to correct a model.
The model was given the following instruction and responded incorrectly.
Instruction: {{ vqa_task_error.task_instance.instruction }}
Model Response: {{ vqa_task_error.predictor_response }}
Correct Response: {{ vqa_task_error.task_instance.ground_truth_label }}
Craft training data to improve the model. The model will be trained on the data you provide.
The training data you produce must be valid JSON with the following fields:
- "inferred_weak_skill": A concise to-the-point description of why you think the model responded incorrectly and
how you'll fix it. Produce this first to give yourself a
chance to think.
- "instruction": The instruction you want the model to respond to.
- "image": The description of an image the instruction is about.
- "response": The correct response to the instruction.
When crafting the training data, consider the following:
- the instructions should be similar in style, length, and complexity to the original instruction
- the images should be relevant to the instruction
- the responses should be similar in style, length, and complexity to the original response
- think about what knowledge the model might be missing that would help it answer the question correctly, and
craft your training data to give it that knowledge
- each response should be a logically _correct_ response to the instruction in the context of the image
description
- the training data should be diverse enough to help the model generalize to new examples
Produce no more than {{ num_training_data }} training data examples.
```

图 12：SKILL-LIST 环境（MATH）数据生成策略提示词示例

```
You are an experienced math educator and your task is to create training data for improving a model's skills in
solving math problems, especially under the category
of "{{ skill_category }}".
Here are examples of mistakes the model made while solving problems requiring "{{ skill_category }}".
The model was given a math problem and responded incorrectly.
{% for math_task_error in math_task_errors %}
- Problem: {{ math_task_error.task_instance.instruction }}
  - Model Response: {{ math_task_error.predictor_response }}
  - Correct Response: {{ math_task_error.task_instance.ground_truth_label }}
{% endfor %}
You will propose hypotheses about what training data the model needs to improve its skills under "{{
skill_category }}".
For certain skills, the model may not have made any mistakes. In that case, propose hypotheses that will help the
model improve on harder examples of the skill.
The training data you produce must be valid JSON using the provided schema.
Here are descriptions of the fields in the schema:
- "inferred_weak_skill": A concise description of the skill under "{{ skill_category }}" that the model is weak
at, and what kind of (problem, response) data will help the
model improve.
- "problem": The math problem you want the model to solve. Ensure this is valid LaTeX that is properly escaped
for representation as a string in Python.
- "chain_of_thought": A step-by-step explanation of how the model should solve the problem. Ensure this is valid
LaTeX that is properly escaped for representation as
a string in Python.
- "final_answer": The final answer to the problem as a LaTeX string. For example '17' or '\\frac{1}{2} or
`\\matrix{1 & 2 \\cr 3 & 4}`. Do not write a sentence here, just
the answer.
Produce {{ num_hypotheses }} hypotheses.
For each hypothesis and weak skill, produce {{ num_data_specs }} specifications for training data.
```

B.3 数据生成细节

对所有任务，我们使用验证准确率判断何时终止回合。达到固定迭代次数时终止，并从策略训练过的所有学生中按验证准确率选择最佳学生。

GQA。使用 stabilityai/sdxl-turbo，4 个推理步，生成 1024 × 1024 图像。在 SKILL-LIST 环境中，基线策略在 3 轮迭代后耗尽数据预算，产生 7.5k 数据点。在 SKILL-TREE 环境中，基线策略回合在 20 步后、约 3k 数据点时产生表现最好的学生。在 OPEN-ENDED 环境中，基线策略回合在 5 步、约 3k 数据点后产生最佳学生。

MATH。数学沿用 GQA 的终止标准。SKILL-LIST 环境中，基线策略产生约 2.1k 数据点，并在 4 轮迭代后得到最佳学生。SKILL-TREE 环境中，基线策略在 20 步内产生 2.8k 数据点，得到最佳学生。OPEN-ENDED 环境中，基线策略需要 10 轮迭代、752 个数据点才产生最佳学生。

LiveCodeBench。对 LiveCodeBench，我们先生成题目，再让 LLM 求解。终止标准与其他设置相同。OPEN-ENDED 环境中，基线策略产生 1.6k 数据点，5 轮迭代后得到最佳学生。SKILL-LIST 环境中，基线策略产生 675 个数据点，11 轮迭代后得到最佳学生。SKILL-TREE 环境中，基线策略产生 3138 个数据点，10 轮迭代后得到最佳学生。

B.4 资源成本

表 7 列出各环境与领域的 token 成本与 GPU 时间。“Coding”领域在 token 支出上最贵，“Multimodal”领域在 GPU 时间上最贵。此外，我们表明可通过更换教师模型降低总体成本。具体而言，我们用 GPT-4o-mini（比 GPT-4o 便宜）作为驱动数据生成智能体的 LLM 做实验，结果见表 6。即使模型缩小，我们仍持续看到性能提升。总体而言，我们预期随着教师模型规模减小，性能提升仍为正但会下降。

表 7：各环境与领域每轮迭代的 token 支出与 GPU 时间。 OPEN-ENDED 环境通常最便宜。最贵的领域是 LiveCodeBench，不过 OPEN-ENDED 的代码实验每次运行成本低于 3 美元。即使只用一张 A6000 GPU，大多数环境的训练时间通常也低于 30 分钟。

领域	环境	Token 数	成本 $（GPT-4o-mini）	成本 $（GPT-4o）	GPU 分钟/迭代
Math	OPEN-ENDED	173,234	0.10	1.73	24
Math	SKILL-LIST	318,528	0.19	3.19	24
Math	SKILL-TREE	355,033	0.21	3.55	16
Coding	OPEN-ENDED	279,304	0.17	2.79	16
Coding	SKILL-LIST	497,787	0.30	4.98	16
Coding	SKILL-TREE	967,610	0.58	9.68	16
Multimodal	OPEN-ENDED	25,073	0.02	0.25	37
Multimodal	SKILL-LIST	82,419	0.05	0.82	134
Multimodal	SKILL-TREE	33,991	0.02	0.34	78

B.5 技能森林细节

技能森林（用于 SKILL-TREE 环境）是一种层级结构，表示各领域的技能与子技能。它以逐步细化的粒度建模学生模型的技能熟练度。森林中的每棵树对应一个高层技能领域，并为每个子技能包含以下关键信息：

1. **数据分配：** 分配给每个子技能的训练数据量。
2. **训练数据表现：** 学生模型在每个子技能训练数据上的表现。
3. **技能名：** 技能的名称。
4. **子技能：** 该技能的子技能列表，初始为空。

森林通过两种机制演化：1. **生长：** 向树中添加新子技能。2. **重新平衡：** 依据表现调整已有子技能的数据分配。这两种动作在概念上对应 explore 与 exploit 技能树。该结构使我们能够表示细粒度技能、为数据生成分配资源、跟踪学生模型进展，并在数据生产过程中优先处理技能。

B.6 SKILL-TREE 策略

我们开发了一个作为表 2 中“With State”基线策略的策略，目标是生长并平衡技能树。它分两个阶段运行。**生长阶段：** 策略在 explore 与 exploit 动作之间交替，直到达到预定的最大子技能数。explore 期间向树中添加新子技能；exploit 期间把数据分配重置为零，为下一次 explore 做准备。**填充阶段：** 达到最大子技能数后，策略切换为纯 exploit 策略，计算并执行动作，逐步为每个子技能分配数据，目标是达到每个子技能的指定最大数据量。策略遵守单次动作的最大分配上限，持续直到所有子技能达到其数据容量。

B.7 验证与测试划分

对 GQA，我们通过对验证集与 testdev 集反复做平衡分层采样来创建验证/测试划分。具体而言，遵循 Gupta & Kembhavi (2023)，从 GQA 的 100 种问题类型中各采样 5 个问题。对 MATH，我们通过对 MATH 测试集在所有层级与主题上做平衡分层采样来创建验证集，每组选 50 个；测试使用 MATH 官方测试集。对

LiveCodeBench，我们选择所有出现在第 2 版但不在第 1 版中的题目作为验证集，整个第 1 版作为测试集；这导致验证集较小，只有 100 道题。

C 定性示例

生成的技能树与样例。 图 13、14、15 分别展示为 GQA、MATH、LiveCodeBench 发现的技能与子技能的定性示例。对每个子技能，我们还展示教师模型生成的示例数据点。注意这些数据点是完整生成的，包括图 13 中的图像。

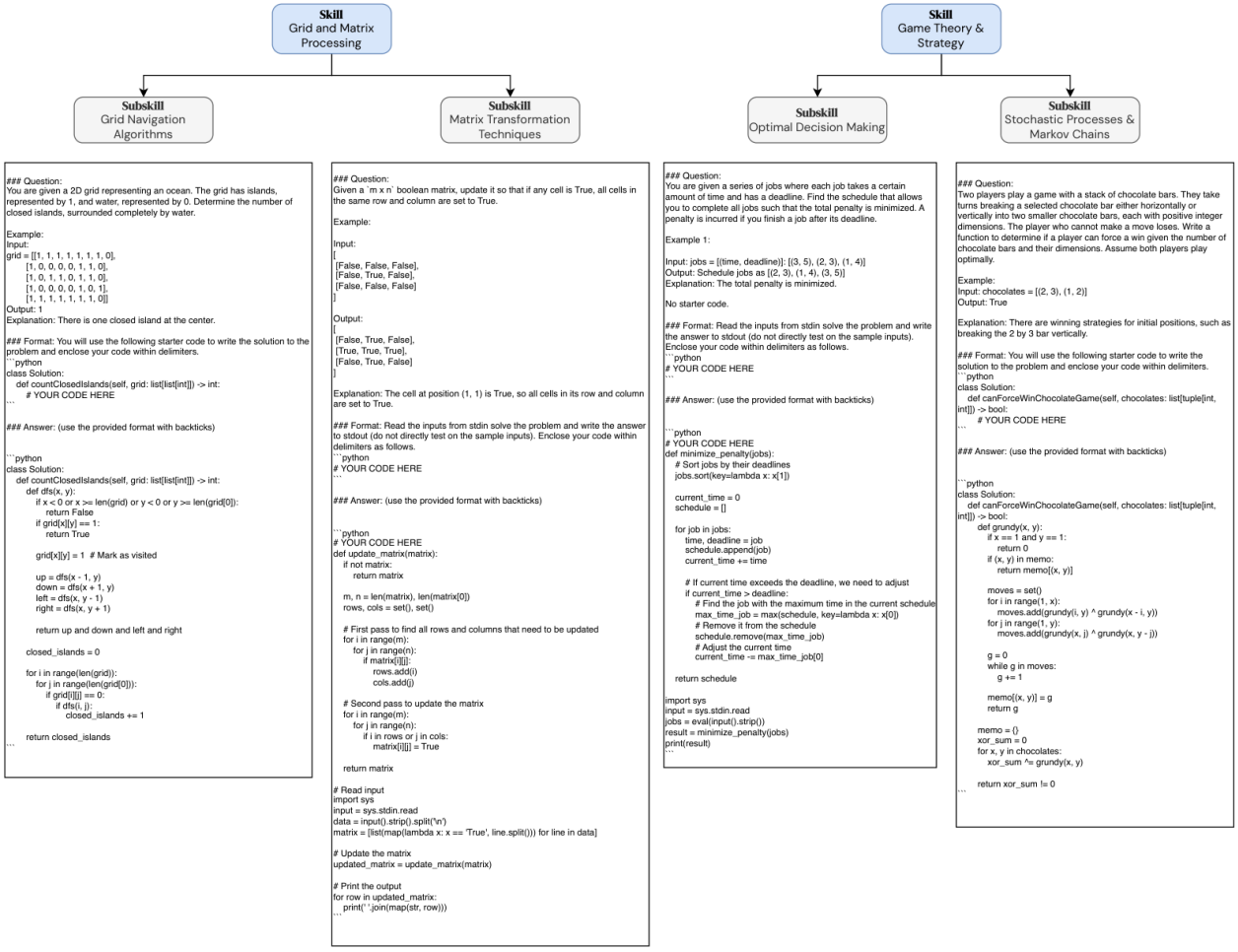


图 15：LiveCodeBench 技能森林的部分视图。图中展示 15 个已发现技能中的 2 个。对技能树中的每个技能，展示 2 个子技能及该子技能的 1 个生成数据示例。

训练前后的模型预测。 图 17 展示在生成数据上训练如何改变 PaliGemma-3B 学生回答的定性示例（见第 3.3 节）。

D 生成训练数据上的准确率

我们分析学生在教师连续迭代生成的数据上的表现，绘制于图 18。具体而言，对 Math 与 Multimodal 环境，我们绘制第 n 轮的学生在第 $n+1$ 轮生成数据上的准确率；也就是说，数据已生成但学生尚未见过。如图 18 所示，学生在教师连续迭代生成的数据上逐步提升。这表明教师产生的数据跨迭代保持一致，学生正逐步适应目标数据分布。然而，教师产生的数据分布足够复杂，提升既非单调，也无法在单轮内完成。

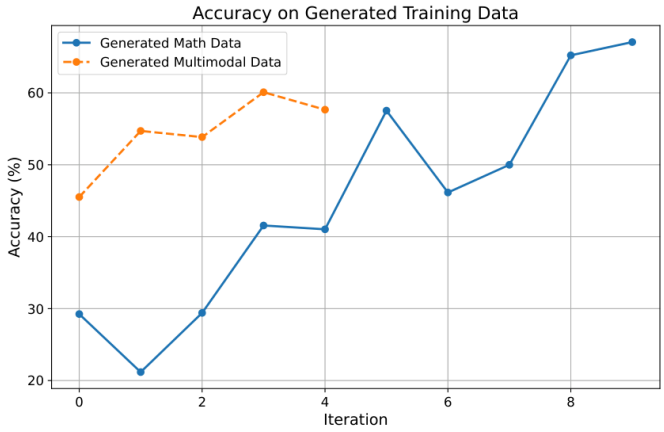


图 18：随着迭代推进，学生对教师生成数据越来越熟练。这里绘制第 n 轮学生在第 n+1 轮教师生成数据上的准确率，这些数据学生尚未见过。

E 消融：区分额外训练与额外数据的影响

为证实学生表现的提升来自新增数据点而非训练不足，我们取数据的一个子集并增加 epoch 数，使学生只收到部分新增数据，但 epoch 数与在完整数据上训练相当。例如，若学生通常用 1000 个生成数据点训练 10 个 epoch，我们取第一次数据生成迭代的数据（例如可能包含 200 个训练数据点），训练一个替代学生 $1000/200 \times 10 = 50$ 个 epoch，以分离生成训练数据与增加训练 epoch 的影响。结果见表 8。每种情况下，数据更少但 epoch 更多的训练所带来的提升都显著小于数据更多、epoch 更少的训练，说明提升来自数据而非更多训练。事实上，在没有新数据的情况下延长训练通常会损害表现——新鲜数据至关重要。这凸显了研究数据生成的重要性，因为数据生成是获得新鲜数据的少数途径之一。

表 8：用更少数据训练更多 epoch 的提升显著小于用更多数据训练更少 epoch，说明提升来自数据而非更多训练。

设置	多模态（GQA）数据 / Epochs / 准确率	MATH 数据 / Epochs / 准确率	Coding（LiveCodeBench）数据 / Epochs / 准确率
教学前	- / - / 44.18	- / - / 15.78	- / - / 16.50
更少数据 / 更长训练	20% / 15 / 42.79	10% / 30 / 13.98	20% / 15 / 15.00
更多数据 / 标准训练	100% / 3 / 47.90	100% / 3 / 23.44	100% / 3 / 18.91